

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
CƠ SỞ TẠI THÀNH PHỐ HỒ CHÍ MINH
KHOA KỸ THUẬT ĐIỆN TỬ 2

BÁO CÁO TỔNG KẾT
ĐỀ TÀI NGHIÊN CỨU KHOA HỌC SINH VIÊN
NĂM HỌC 2023 – 2024

THIẾT KẾ XE DÒ LINE
GIẢI MÃ MÊ CUNG
MÃ SỐ ĐỀ TÀI: 31-SV-2023-ĐT2

Thuộc nhóm ngành khoa học: Công Nghệ Kỹ Thuật Điện – Điện Tử

Sinh viên thực hiện:

Trần Quốc Cường	N21DCDT016
Hoàng Minh Vương	N21DCDT105
Nguyễn Việt Thành	N21DCDT083
Nguyễn Thái Vinh	N21DCDT102
Lớp : D21CQDT01-N	

Giáo viên hướng dẫn: ThS. Nguyễn Lan Anh

TP.HCM, tháng 10 năm 2023

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
CƠ SỞ TẠI THÀNH PHỐ HỒ CHÍ MINH
KHOA KỸ THUẬT ĐIỆN TỬ 2

BÁO CÁO TỔNG KẾT
ĐỀ TÀI NGHIÊN CỨU KHOA HỌC SINH VIÊN
NĂM HỌC 2023 – 2024

THIẾT KẾ XE DÒ LINE
GIẢI MÃ MÊ CUNG
MÃ SỐ ĐỀ TÀI: 31-SV-2023-ĐT2

Thuộc nhóm ngành khoa học: Công Nghệ Kỹ Thuật Điện – Điện Tử

Sinh viên thực hiện:

Trần Quốc Cường	N21DCDT016
Hoàng Minh Vương	N21DCDT105
Nguyễn Việt Thành	N21DCDT083
Nguyễn Thái Vinh	N21DCDT102
Lớp : D21CQDT01-N	

Giáo viên hướng dẫn: ThS. Nguyễn Lan Anh

TP.HCM, tháng 10 năm 2023

MỤC LỤC

MỞ ĐẦU	4
1. Giới thiệu:	4
2. Mục đích đề tài:	4
3. Lý do chọn đề tài:	4
4. Đối tượng nghiên cứu.....	4
5. Các bước thực hiện cơ bản:	4
CHƯƠNG 1: CƠ SỞ LÝ THUYẾT	5
1.1 Sơ lược về Arduino:	5
1.1.1 ARDUINO NANO:.....	5
1.1.2 Các thông số kỹ thuật ARDUINO NANO:.....	6
1.2 Cảm biến và module chức năng:	6
1.2.1 Cảm biến hồng ngoại:	6
1.2.1.1 Cảm biến line 8 kênh QTR8A-TH063:	6
1.2.1.2 Cảm biến hồng ngoại TCRT5000:.....	8
1.2.1.3 Nguyên lý hoạt động cảm biến hồng ngoại:.....	9
1.2.2 Module L298N:.....	10
1.2.3 LM2596S Mạch giảm áp 3A:	11
1.5 Mạch mở rộng Arduino Nano Shield V3.0:.....	12
CHƯƠNG 2: PHÂN TÍCH THIẾT KẾ HỆ THỐNG	13
2.1 Đặc tả hệ thống:	13
2.2 Sơ đồ hệ thống:	14
2.2.1 Sơ đồ khối hoạt động của hệ thống:	14
2.2.2 Sơ đồ khối nối các phần cứng:.....	15
2.2.3 Sơ đồ khối hoạt động của code:.....	16
2.2.3.1 Sơ đồ khối hàm void setup():	16
2.2.3.2 Sơ đồ khối hàm void loop():.....	17
2.2.3.3 Sơ đồ khối hàm indongco():	18
2.2.3.4 Sơ đồ khối hàm incambien():	19
2.2.3.5 Sơ đồ khối Hàm uint8_t digital(uint8_t x):	20
2.2.3.6 Sơ đồ khối Hàm resetcb():.....	21
2.2.3.7 Sơ đồ khối Hàm phandoan():	22
2.2.3.8 Sơ đồ khối Hàm char line():	23
2.2.3.10 Sơ đồ khối Hàm void dongcophai(char x, unsigned int td):	25

2.2.3.11 Sơ đồ khối Hàm void dongco(char x, unsigned int tdt, unsigned int tdp):	27
2.2.3.12 Sơ đồ khối Hàm void theoline():	28
2.2.3.13 Sơ đồ khối Hàm void hanhdong():	29
2.2.3.15 Sơ đồ khối Hàm void rutngan(char x):	31
CHƯƠNG 3: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	32
3.1 Hình ảnh của xe:	32
3.1.1 Hình ảnh xe sau khi hoàn thành:	32
3.7 Hình ảnh xe trong mê cung line:	35
3.2 Kết quả đạt được:	35
3.2.1 Thành viên:	35
3.2.2 Sản phẩm:	35
3.2.3 Đánh giá sản phẩm:	35
3.2.4 Hướng Phát Triển:	35
Phần Phụ Lục:	37

MỞ ĐẦU

1. Giới thiệu:

Ngày nay, sự phát triển của robot điện tử đang đem lại những tiến bộ đáng kể trong lĩnh vực công nghệ và ứng dụng. Từ những robot đơn giản chỉ có khả năng di chuyển và thực hiện các nhiệm vụ cơ bản, cho đến những robot phức tạp có khả năng tương tác với con người và thực hiện các tác vụ phức tạp, lĩnh vực này đang tiến xa hơn trong việc đáp ứng nhu cầu và giải quyết các vấn đề hiện tại.

Robot điện tử ngày càng có thể thực hiện các nhiệm vụ hữu ích trong cuộc sống hàng ngày và các lĩnh vực khác, thu hút được rất nhiều sự đầu tư và nghiên cứu. Robot điện tử cũng có rất nhiều loại, trong đó có 1 loại dễ dàng nghiên cứu, phát triển và ứng dụng vào trong cuộc sống là robot dò line giải mã mê cung. Việc phát triển loại robot này sẽ phục vụ đắc lực cho con người.

2. Mục đích đề tài:

Học tập và vận dụng kiến thức đã học trong trường để áp dụng tạo nên sản phẩm thực tế.

Trải nghiệm làm sản phẩm thực tế.

Rèn luyện học tập và nghiên cứu.

3. Lý do chọn đề tài:

Phù hợp với trình độ của thành viên trong nhóm, và khả năng làm được đề tài của thành viên.

4. Đối tượng nghiên cứu

Mạch xử lý điều khiển đọc cảm biến và điều khiển động cơ sử dụng vi điều khiển Atmega 328

Giải thuật

5. Các bước thực hiện cơ bản:

Chuẩn bị đầy đủ các thiết bị để chế tạo xe robot, chế tạo khung xe (phải chắc chắn) và bố trí 1 cách chính xác các động cơ và mạch điện tử lên xe.

Cuối cùng là lập trình cho xe robot hoạt động.

Viết báo cáo.

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT

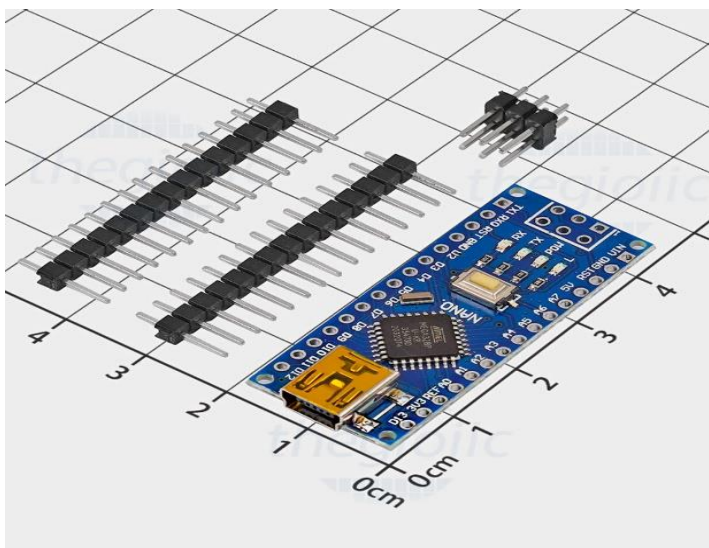
1.1 Sơ lược về Arduino:

Arduino là một nền tảng điện tử mã nguồn mở dựa trên phần cứng và phần mềm thân thiện dễ sử dụng. Các board Arduino có khả năng đọc các tín hiệu đầu vào từ các cảm biến, nút nhấn – và biến chúng thành tín hiệu đầu ra-kích hoạt động cơ, bật đèn LED. Bạn có thể chỉ định cho board của mình làm gì bằng cách gửi một tập hợp các lệnh tới vi điều khiển trên board. Để làm điều này, bạn sử dụng ngôn ngữ lập trình Arduino và phần mềm Arduino (IDE). [1]

Qua các năm, Arduino đã trở thành bộ não của hàng ngàn các dự án, từ những đồ vật hàng ngày đến các thiết bị khoa học phức tạp. Một cộng đồng toàn cầu của những người sáng tạo – sinh viên, người làm trong lĩnh vực liên quan, các chuyên gia đã đóng góp vào nền tảng mã nguồn mở Arduino, tạo nên sự tích lũy một lượng kiến thức khổng lồ có ích cho cả người mới bắt đầu và các chuyên gia. [1]

Arduino ra đời tại Ivrea Interaction Design Institute như một công cụ dễ sử dụng để tạo ra nguyên mẫu một cách nhanh chóng, dành cho sinh viên mới bắt đầu về điện tử và lập trình. Ngay khi nó trở nên thành một cộng đồng đông đảo, board Arduino bắt đầu thay đổi để thích ứng với các nhu cầu và thách thức mới, phân biệt sản phẩm của họ từ các board 8-bit đơn giản đến các sản phẩm dành cho ứng dụng IoT, thiết bị đeo, in 3D và môi trường nhúng. [1]

1.1.1 ARDUINO NANO:



Hình 1.1: Arduino Nano

<https://www.thegioiic.com/arduino-nano-atmega328-v3>

Arduino Nano là một board mạch dựa trên Atmega328 (Arduino Nano 3.x) nhỏ gọn, và dễ dàng lắp ráp với Breadboard. Nó có chức năng tương đương với Arduino

Duemilanove nhưng trong một dạng khác. Nó chỉ thiếu một jack nguồn DC và hoạt động với cáp Mini-B USB thay vì cáp tiêu chuẩn. [2]

1.1.2 Các thông số kỹ thuật ARDUINO NANO:

Vi điều khiển chính: ATmega328

Điện áp hoạt động: 5VDC

Điện áp vào: 7 ~ 12VDC

Điện áp vào giới hạn: 6 ~ 20 VDC

Số chân Digital: 14 (6 chân PWM)

Số chân vào Analog: 8

Dòng DC trên mỗi chân: 40mA

Dòng DC trên chân 3.3V: 50mA

Bộ nhớ Flash: 32 KB (2KB dùng cho bootloader)

SRAM: 2 KB

EEPROM: 1KB

Tần số xung clock: 16 MHz [3]

1.2 Cảm biến và module chức năng:

Trong xe do line giải mã mê cung có 2 loại cảm biến hồng ngoại, một loại là 8 kênh, và một loại là module hồng ngoại riêng lẻ. Và có module điều khiển động cơ.

1.2.1 Cảm biến hồng ngoại:

1.2.1.1 Cảm biến line 8 kênh QTR8A-TH063:



Hình 1.2: Cảm biến line 8 kênh QTR8A-TH063

<https://imaker.vn/cam-bien-do-line-8-kenh-qtr-8a>

Cảm biến dò line 8 kênh QTR8A-TH063 là một loại cảm biến phản xạ hồng ngoại được sử dụng phổ biến trong các ứng dụng robot và xe tự hành. Cảm biến được thiết

kể để phát hiện vị trí của đường line hoặc các vật thể có độ tương phản khác nhau trên bề mặt.

Cảm biến có kích thước nhỏ gọn và dễ dàng lắp đặt. Được sử dụng để dễ dàng phân biệt các màu sắc có độ tương phản cao.

QTR8A-TH063 có 8 kênh dò, cho phép cảm biến phát hiện và đọc dữ liệu, cho phép nó phát hiện và đọc dữ liệu từ 8 đường line cùng một lúc. Điều này cải thiện độ chính xác và đáng tin cậy của việc phát hiện line.

Tình năng:

Phát hiện được độ tương phản của các màu sắc khác nhau như trắng và đen.

Thông số kỹ thuật:

Kích thước: 2,95" x 0,5" x 0,125" (không cài đặt chân tiêu đề)

Điện áp hoạt động: 3,3-5,0 V

Dòng cung cấp: 100 mA

Định dạng đầu ra: 8 điện áp analog

Dải điện áp đầu ra: 0 V đến điện áp cung cấp

Khoảng cách phát hiện tối ưu: 0,125" (3 mm)

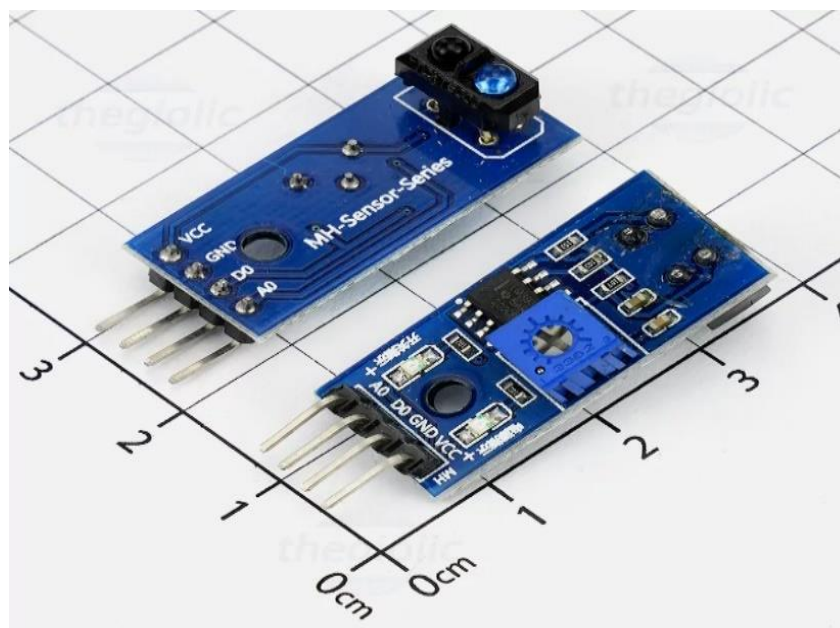
Khoảng cách phát hiện tối đa được đề nghị: 0,25" (6 mm)

Trọng lượng không có chân tiêu đề: 0,11 oz (3,09 g)

[4]

Cảm biến QTR8A-TH063 được thiết kế để hoạt động trong môi trường ánh sáng mạnh và ánh sáng yếu, nhưng độ chính xác có thể bị ảnh hưởng bởi điều kiện ánh sáng. Do đó, cảm biến thường được sử dụng trong nhà hoặc trong điều kiện ánh sáng ổn định.

1.2.1.2 Cảm biến hồng ngoại TCRT5000:



Hình 1.3: Cảm biến hồng ngoại TCRT5000

<https://www.thegioiic.com/tcrt5000-cam-bien-do-line-khoang-cach-0-3cm-ngo-ra-ttl>

TCRT5000 Cảm Biến Dò Line sử dụng mắt thu phát hồng ngoại TCRT5000 có độ nhạy cao, giúp phát hiện được độ tương phản của các màu sắc khác nhau như trắng và đen. Trên mạch sử dụng chip so sánh LM393 hoạt động rất ổn định, có lỗi bắt ốc thuận tiện. Được ứng dụng trong nhiều lĩnh vực khác nhau như: đo khoảng cách, phát hiện chướng ngại vật, dò đường (dò line), ...

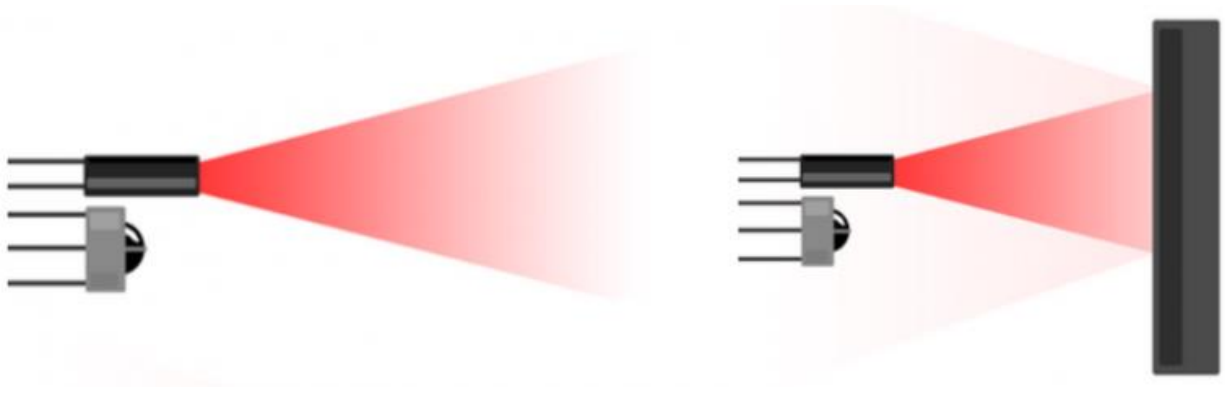
Tính năng:

Phát hiện được độ tương phản của các màu sắc khác nhau như trắng và đen.

Thông số kỹ thuật:

Module có thể dò khoảng cách 0~3 cm
Điện áp làm việc: 3,3 ~ 5 VDC
Dòng tiêu thụ: 10mA
Kích thước mạch: 3,2 x 1,4 cm
Khoảng cách phát hiện phản xạ: 1 ~ 25 mm
Mức tín hiệu ngõ ra: TTL [5]

1.2.1.3 Nguyên lý hoạt động cảm biến hồng ngoại:

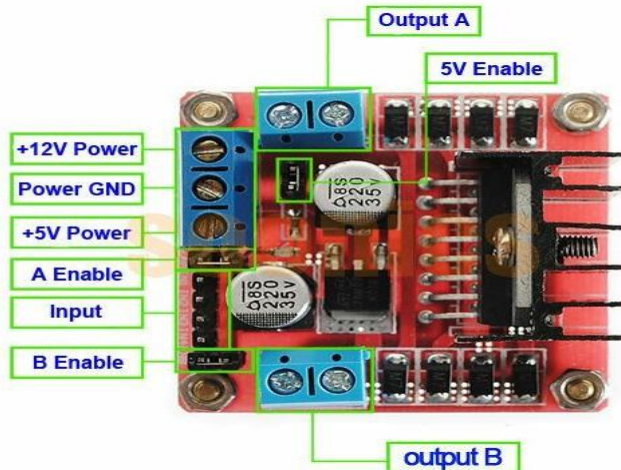


Hình 1.4: Nguyên lý hoạt động cảm biến hồng ngoại

<https://thietbikythuat.com.vn/nguyen-ly-hoat-dong-cua-cam-bien-hong-ngoai-la-gi/>

Cảm biến hồng ngoại hoạt động bằng cách sử dụng một cảm biến ánh sáng cụ thể nào đó để phát hiện ra bước sóng ánh sáng chọn trong phổ hồng ngoại (IR). Bằng cách sử dụng đèn LED để tạo ra ánh sáng có cùng bước sóng với cảm biến đang tìm kiếm. Khi ánh sáng đèn LED gặp vật thể thì ánh sáng chiếu từ đèn LED sẽ bật ra khỏi vật thể và đi vào cảm biến ánh sáng. Điều này dẫn đến một bước nhảy khá lớn về cường độ. [6]

1.2.2 Module L298N:



Hình 1.5: Module L298N

<http://arduino.vn/bai-viet/893-cach-dung-module-dieu-khien-dong-co-l298n-cau-h-de-dieu-khien-dong-co-dc>

Mạch điều khiển động cơ L298 DC Motor Driver có khả năng điều khiển 2 động cơ DC, dòng tối đa 2A mỗi động cơ, mạch tích hợp diod bảo vệ và IC nguồn 7805 giúp cấp nguồn 5VDC cho các module khác (lưu ý chỉ sử dụng nguồn 5V này nếu nguồn cấp VCC < 12VDC).

Tính năng:

Dùng để điều khiển động cơ DC một cách dễ dàng.

Thông số kỹ thuật:

Driver: L298N tích hợp hai mạch cầu H.

Điện áp điều khiển: +5 V ~ +12 V

Dòng tối đa cho mỗi cầu H là: 2A ($\Rightarrow$ 2A cho mỗi motor)

Điện áp của tín hiệu điều khiển: +5 V ~ +7 V

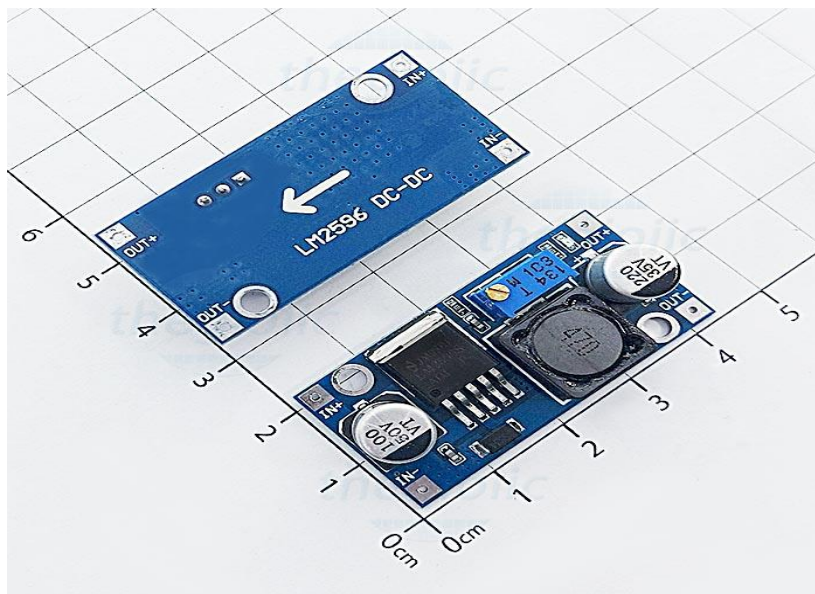
Dòng của tín hiệu điều khiển: 0 ~ 36mA (Arduino có thể chơi đến 40mA nên khỏe re nhé các bạn)

Công suất hao phí: 20W (khi nhiệt độ $T = 75^{\circ}\text{C}$)

Nhiệt độ bảo quản: $-25^{\circ}\text{C} \sim +130^{\circ}\text{C}$

[7]

1.2.3 LM2596S Mạch giảm áp 3A:



Hình 1.6: LM2596S Mạch giảm áp 3A

<https://www.thegioiic.com/lm2596s-mach-giam-ap-3a>

LM2596S là một mạch giảm áp có khả năng điều khiển tải lên đến 3A, mạch có dải điện áp đầu vào rộng từ 3.2 đến 40VDC, điện áp đầu ra từ 1.25 đến 40VDC có thể điều chỉnh dễ dàng thông qua biến trở tinh chỉnh được tích hợp trên mạch.

Tính năng:

Dùng để giảm áp hoặc ổn định áp ra khỏi nguồn.

Thông số kỹ thuật:

Tên: LM2596S DC-DC step down module

Điện áp vào: 3.2V~40V

Điện áp ra: 1.25V~35V

Dòng ra: 3A (Max)

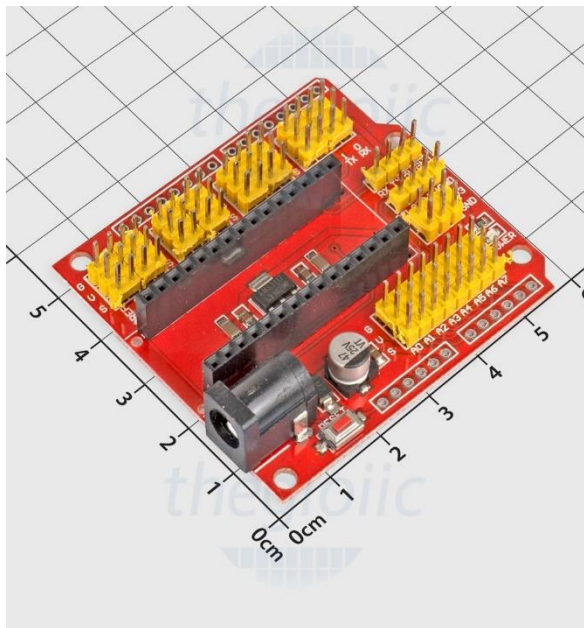
Hiệu suất chuyển đổi: 92% (Max)

Tần số dao động: 65KHz

Nhiệt độ: -45°C~ +85°C

Kích thước: 43mm * 21mm * 14mm [8]

1.5 Mạch mở rộng Arduino Nano Shield V3.0:



Hình 1.7: Mạch mở rộng Arduino Nano Shield V3.0

<https://www.thegioiic.com/mach-mo-rong-arduino-nano-shield-v3-0>

Mạch mở rộng này được thiết kế riêng cho Arduino Nano, đưa ra tất cả các cổng IO số và IO tương tự, mỗi cổng có nguồn giao tiếp âm và dương tiêu chuẩn, đưa ra kết nối giao tiếp I2C với Board chính, mang lại thuận tiện khi kết nối.

Thêm cổng cấp nguồn DC. Với Arduino Nano thường chỉ hỗ trợ cấp nguồn qua USB với dòng cung cấp là 50mA (như vậy sử dụng với thiết bị cần nguồn dòng cao hơn là không đủ). Vì vậy cổng cấp nguồn DC mở rộng sẽ đảm bảo cho sự làm việc ổn định của thiết bị.

- **Tính năng:**

Mạch Mở Rộng Arduino Nano Shield V3.0 là một board mở rộng đa chức năng cho Arduino Nano Atmega328. Nó giải quyết bài toán về chân kết nối khi kết nối Arduino Nano với các cảm biến.

- **Thông số kỹ thuật:**

14 ngõ I/O (servo type with GND, power and signal)

8 analog Pin with power output and GND

6 PWM Pin

5I2C expansion Pin

AREF output

3,3V output[9]

CHƯƠNG 2: PHÂN TÍCH THIẾT KẾ HỆ THỐNG

2.1 Đặc tả hệ thống:

Tính năng của sản phẩm: dò line di chuyển theo line từ đó giải mã mê cung line.

Hoạt động của sản phẩm: Đầu tiên cảm biến QTR8A-TH063 và cảm biến TCRT5000 sẽ phát hiện line, dò line và gửi tín hiệu về trung tâm Arduino Nano thông qua các chân mở rộng của Arduino Nano Shield.

Sau đó Arduino Nano nhận được các tín hiệu và xử lý nhận dạng các line, tìm hướng ra cho mê cung, điều khiển xe theo ý muốn. Từ đó đưa ra các tín hiệu để điều khiển module L298N.

Sau đó module L298N nhận được các tín hiệu rồi xử lý và điều khiển hai động cơ theo tốc độ và hướng quay nhận được từ tín hiệu của Arduino Nano.

Cách giải mã mê cung:

Trong trạng thái đầu tiên của xe: Xe sẽ bám theo lề trái để tìm đường ra khỏi mê cung, cứ mỗi lần qua các ngã ba, ngã tư, hoặc đường tiến rẽ trái, đường tiến rẽ phải hoặc đường cắt, xe sẽ đánh dấu lại những hành động xe đã làm như: Ngã ba thì xe đã rẽ trái, ngã tư xe đã rẽ trái. Sau khi đánh dấu và rẽ xong, Arduino Nano sẽ xử lý rút gọn mê cung theo quy tắc sau:

.LBR = B

.LBH = R

.RBL = B

.HBL = R

.HBH = B

.LBL = S [10]

(L là rẽ trái, B là quay đầu, R là rẽ phải, H là tiến tới)

Xe sẽ rút ngắn lại mê cung sau khi đi qua ngõ cụt một lần và gặp trường hợp tiếp theo kể bên nó. Lúc đó Arduino nó sẽ rút ngắn mê cung lại từ những gì đã đánh dấu theo quy tắc trên, và lưu lại

Sau khi xe tới đích thì kết thúc trường hợp 1, bắt đầu trường hợp 2.

Trường hợp 2: Quay lại nơi bắt đầu

Lúc này xe dựa vào những thứ đã lưu lại khi giải mã mê cung, từ đó đi ngược lại tới điểm đến. Khi đến nơi bắt đầu. Trạng thái 2 kết thúc và bắt đầu trạng thái 3.

Trạng thái 3: Đi ra khỏi mê cung với đường đã tối ưu

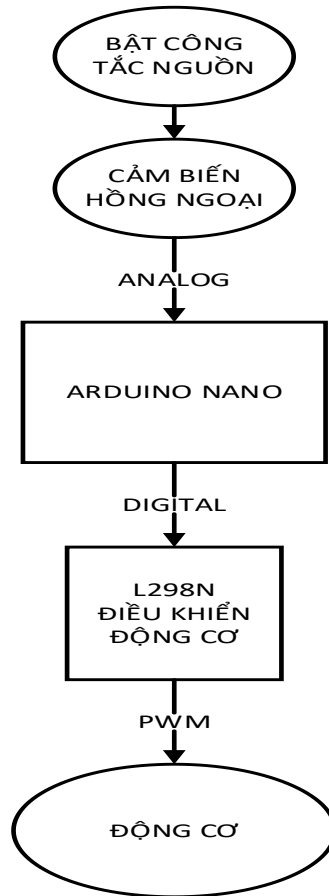
Xe bắt đầu chạy theo ra khỏi mê cung theo đường đã rút gọn. Khi đến đích xe kết thúc trường hợp 3 và dừng lại.

Hoàn thành xong nhiệm vụ của xe.

2.2 Sơ đồ hệ thống:

Trong phần này ta đề cập đến sơ đồ khối nối giữa các phần cứng, sơ đồ khối của cách hoạt động phần cứng kể cả phần hoạt động của code.

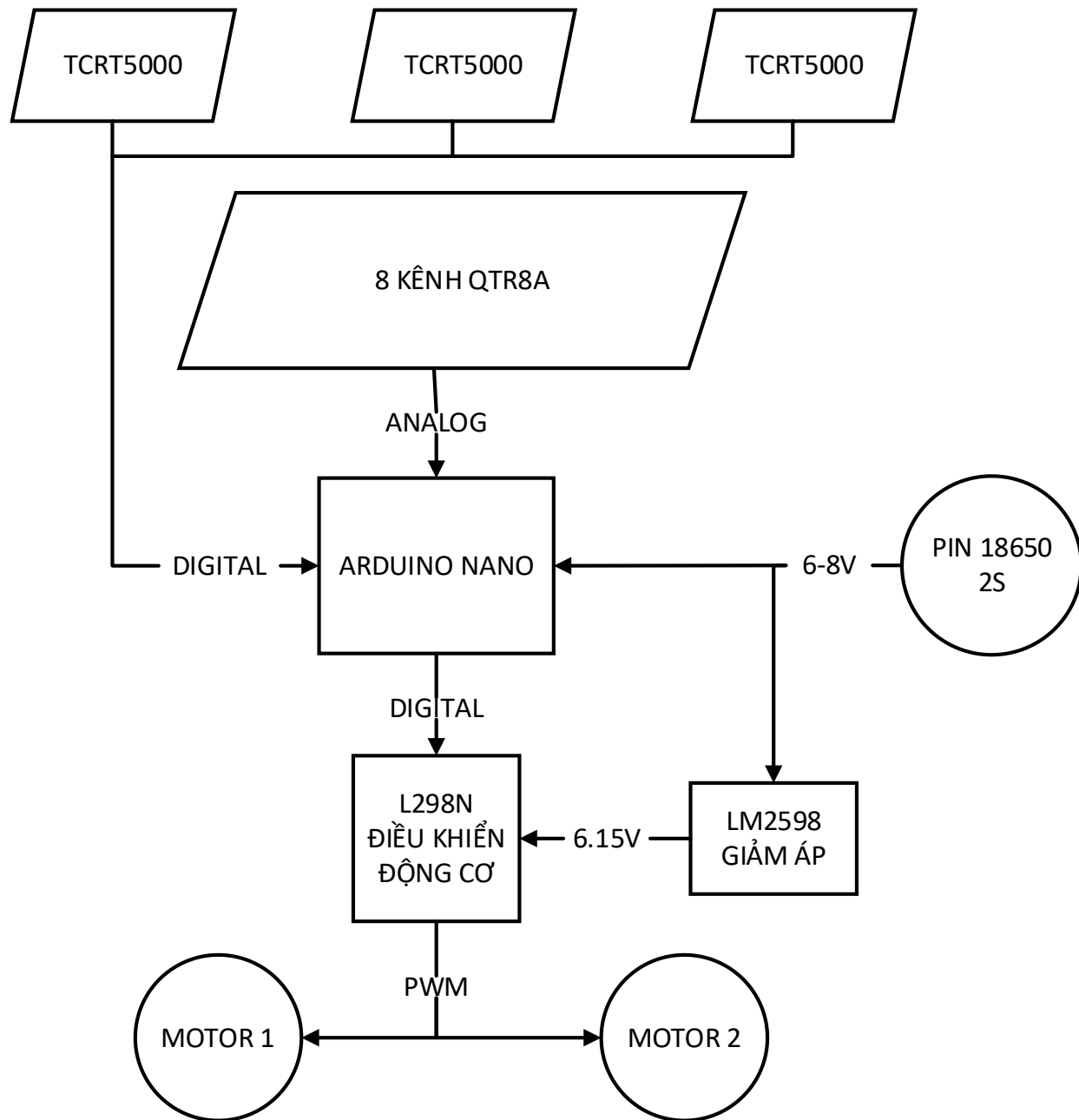
2.2.1 Sơ đồ khối hoạt động của hệ thống:



Hình 2.1 Trình tự hoạt động của của xe

Khi xe được bật công tắc, trình tự hoạt động của xe sẽ hoạt động tuần tự như sơ đồ hình 2.1

2.2.2 Sơ đồ khối nối các phần cứng:

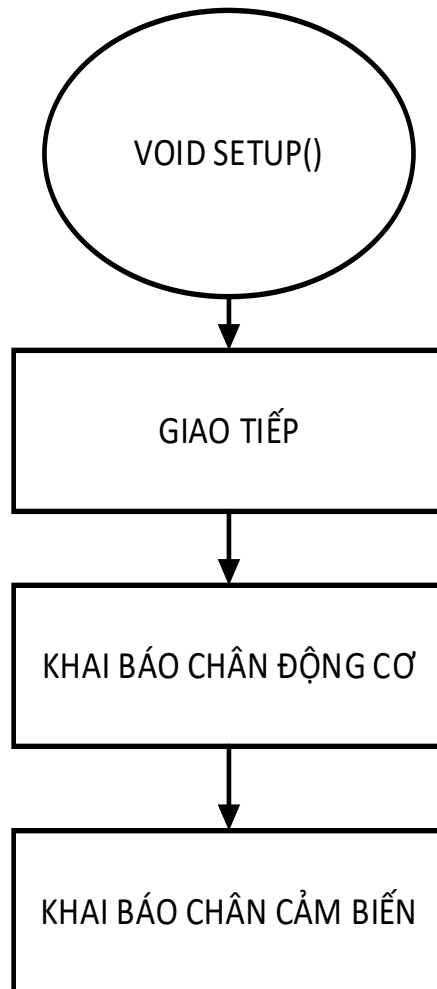


Hình 2.2 Sơ đồ nối giữa các module với nhau

Sơ đồ hình 3.2 biểu thị cho việc các module trong xe được kết nối với nhau như thế nào. Và truyền tín hiệu PWM hay digital hoặc là điện áp để cung cấp nguồn.

2.2.3 Sơ đồ khối hoạt động của code:

2.2.3.1 Sơ đồ khối hàm void setup():



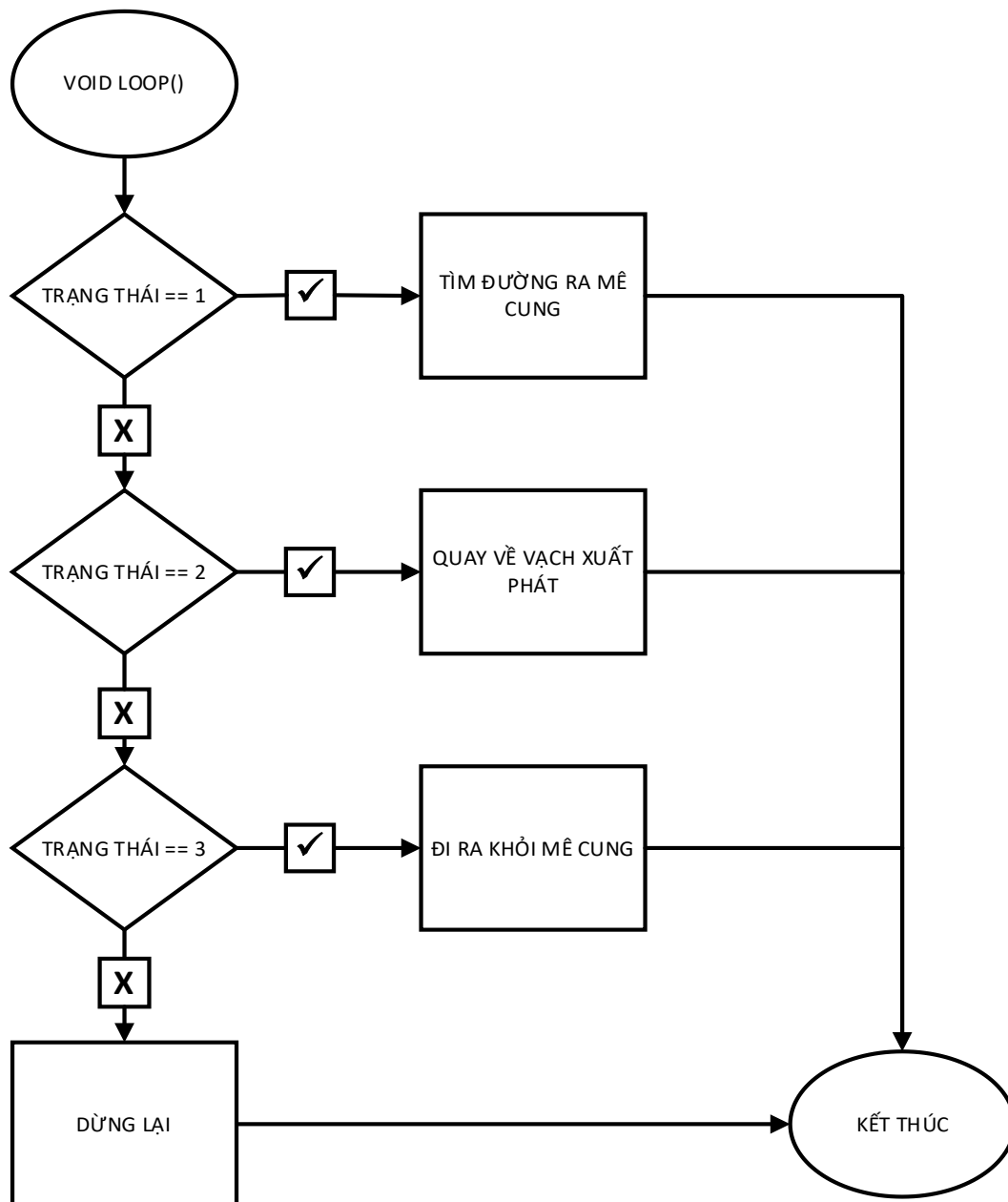
Hình 2.3 Sơ đồ khối void setup()

Trong sơ đồ hình 2.3:

Có khối khai báo chân động cơ: khối này cho biết vi điều khiển biết chân nối module điều khiển động cơ ở trạng thái nào. Chi tiết được đề cập ở mục 2.2.3.3.

Khối khai báo chân cảm biến: Khối này cho biết vi điều khiển biết chân nối với cảm biến ở trạng thái nào. Chi tiết được đề cập ở mục 2.2.3.4

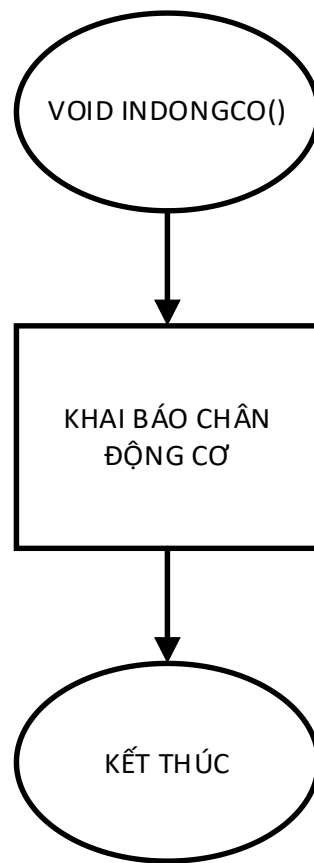
2.2.3.2 Sơ đồ khối hàm void loop():



Hình 2.4 Hàm loop (hàm chính của xe)

Trong khối tìm đường ra khỏi mê cung. Hàm này có một biến nhận được giá trị từ hàm `phandoan`. Đây là một hàm sẽ được đề cập ở mục 2.2.3.7. Sau khi nhận được giá trị thì dựa vào cấu trúc `switch case` có trong khối, từ đó vi điều khiển có thể ra lệnh điều khiển động cơ thông qua hàm `hanhdong()`. Hàm này sẽ được đề cập ở mục 2.2.3.13.

2.2.3.3 Sơ đồ khối hàm indongco():



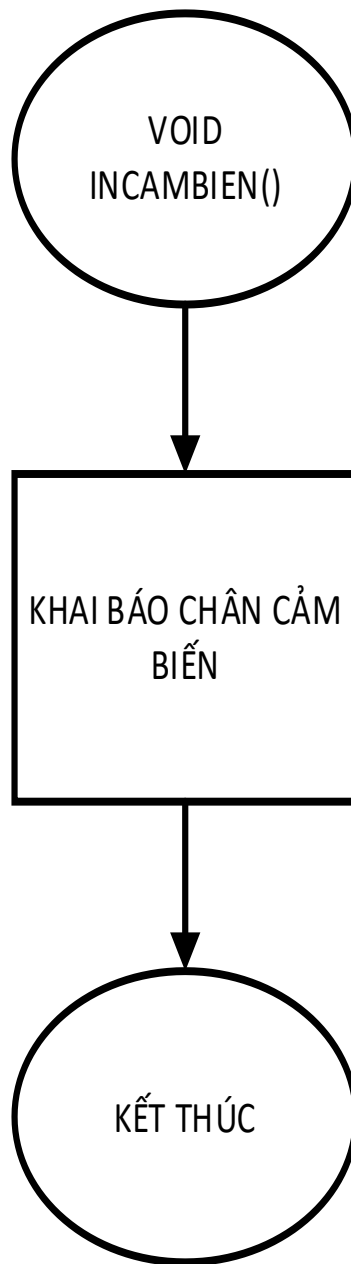
Hình 2.5 Sơ đồ khối hàm indongco()

Hàm này sẽ là một loạt các câu lệnh `pinMode()` giúp cho vi điều khiển xác định các chân nối với module điều khiển động cơ là ở dạng `output()`. Các chân ở trên vi điều khiển được nối với module điều khiển động cơ cụ thể như sau:

Chân 5, 6, 9 cho động cơ bên trái.

Chân 7, 8, 10 cho động cơ bên phải.

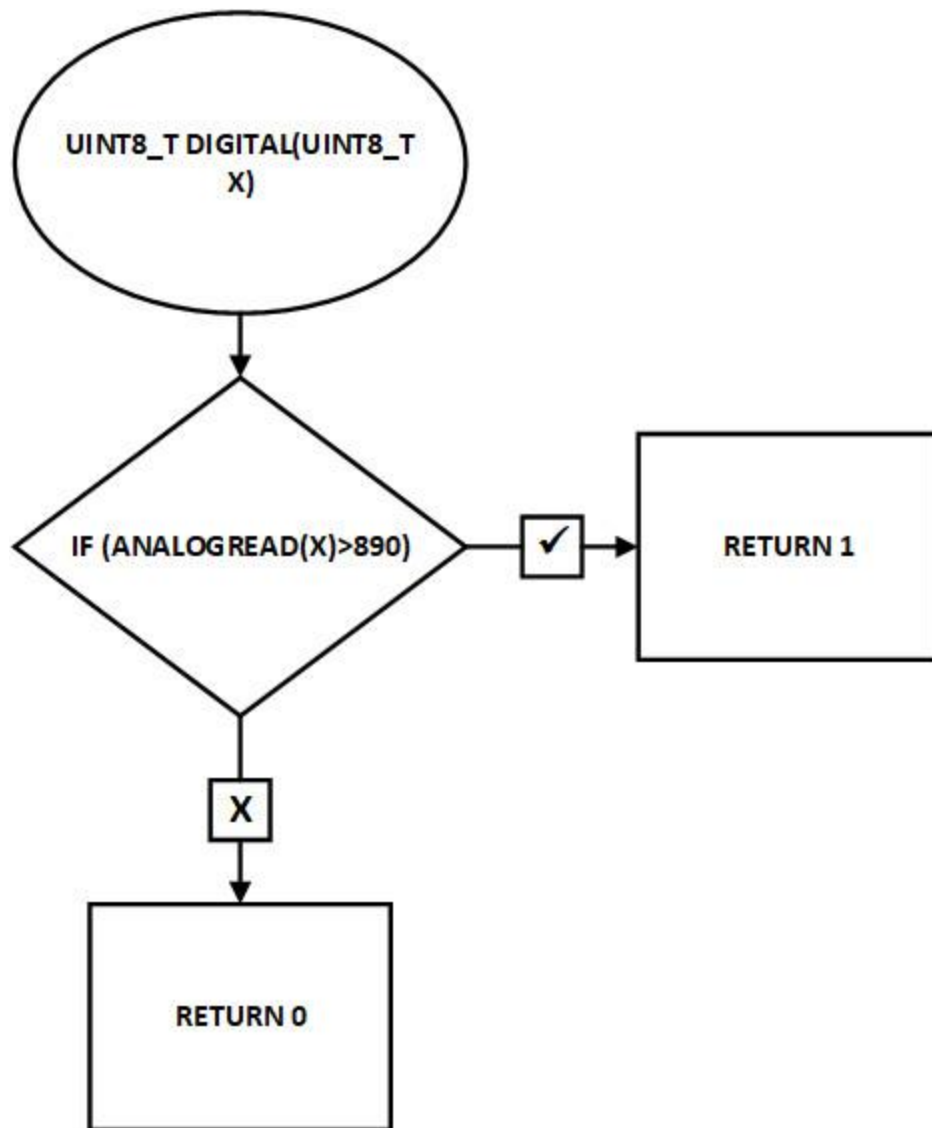
2.2.3.4 Sơ đồ khối hàm incambien():



Hình 2.6 Sơ đồ khối hàm incambien()

Hàm này sẽ là một loạt các câu lệnh `pinMode()` giúp cho vi điều khiển xác định các chân nối với các module cảm biến hồng ngoại là ở dạng `input()`. Các chân ở trên vi điều khiển được nối với module cảm biến hồng ngoại cụ thể như sau: A0, A1, A2, A3, A4, A5, A6, A7, 3, 4, 11. Trong đó chân 3, 4, 11 là cho 3 cảm biến hồng ngoại TCRT5000 riêng biệt.

2.2.3.5 Sơ đồ khối Hàm uint8_t digital(uint8_t x):



Hình 2.7 Sơ đồ khối hàm `uint8_t digital(uint8_t x)`

Hàm này có tác dụng đổi các giá trị analog ở cảm biến dò line 8 kênh được đề cập ở mục 1.2.1. Thành các giá trị ở dạng bool để có thể dễ dàng xử lý.

Hàm này nhận giá trị từ bên trong hàm `resetcb` được đề cập ở mục 2.2.3.6. Nó sẽ cho biết được các chân cần lấy tín hiệu từ đó hàm này sẽ so sánh với số được ghi sẵn do người lập trình quyết định. Và nó sẽ trả về giá trị 1 hoặc 0.

Số do người lập trình quyết định này sẽ dựa trên số lần người lập trình cho xe chạy thử và sửa đi sửa lại số này cho tới khi xe chạy ổn định trên mọi trường hợp được kiểm tra.

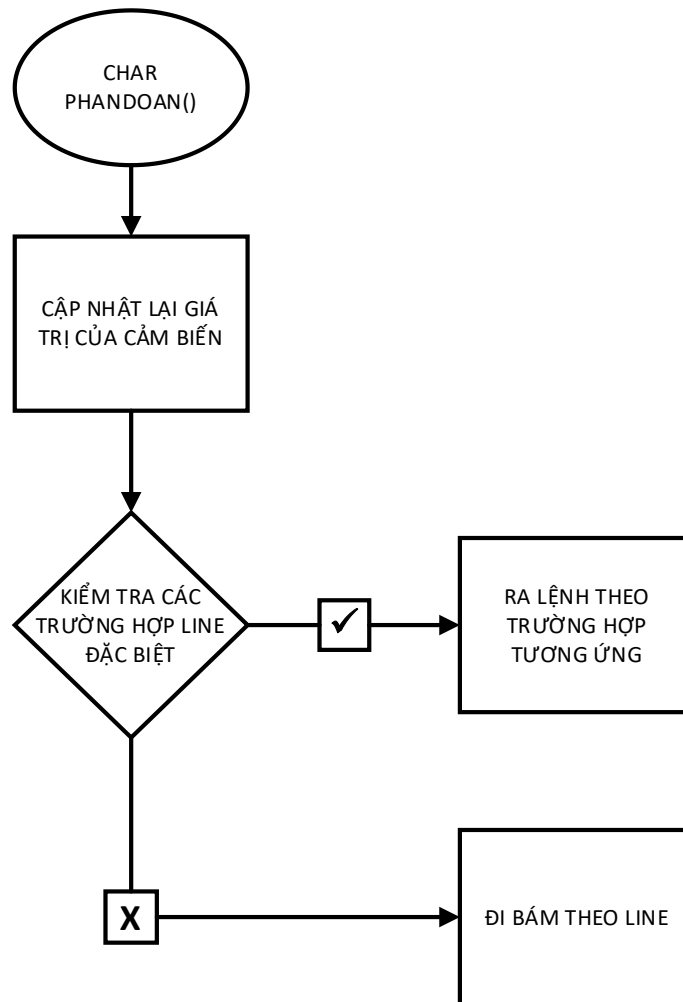
2.2.3.6 Sơ đồ khối Hàm resetcb():



Hình 2.8 Sơ đồ của hàm resetcb()

Hàm này có tác dụng cập nhật lại giá trị của các biến chứa giá trị trả về của cảm biến. Hàm sẽ nhờ vào hàm `uint8_t digital()` đề cập ở mục 2.2.3.5 để cập nhật giá trị của cảm biến trả về dạng analog và nhờ lệnh `digitalRead` đọc các cảm biến ở dạng digital.

2.2.3.7 Sơ đồ khối Hàm phandoan():



Hình 2.9 Sơ đồ khối hàm phandoan()

Lưu ý: dấu tích trong sơ đồ biểu thị cho việc kiểm tra điều kiện đã đúng, dấu x là biểu thị cho việc kiểm tra điều kiện đã sai

Hàm này có tác dụng giúp vi điều khiển nhận biết được các dạng line chẳng hạn như ngã ba, ngã tư, đường cụt hoặc đích đến.

Trong hàm này, đầu tiên hàm sẽ cập nhật lại giá trị của cảm biến bằng hàm `resetcb()` được đề cập ở mục 2.2.3.6. Lúc này dựa vào cấu trúc `if else` kiểm tra các giá trị được cập nhật trước đó để có thể trả về các giá trị như:

‘S’: đường cụt

‘f’: Ngã tư

‘t’: Ngã ba chữ t

‘a’: ngã ba trái

‘d’: ngã ba phải

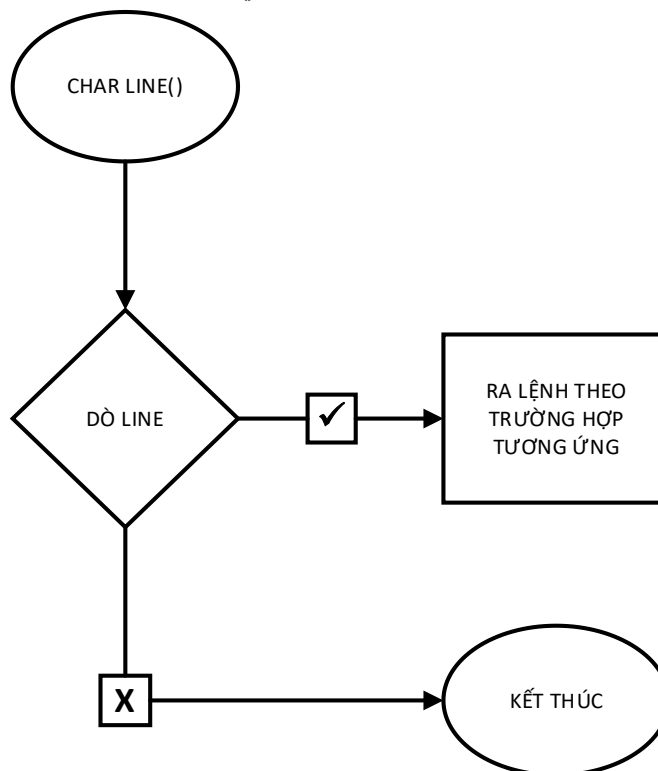
‘l’: ngã trái

‘r’: ngã phải

‘o’: đường cụt

‘h’: biểu thị việc đi theo line

2.2.3.8 Sơ đồ khối Hàm char line():



Hình 2.10 Sơ đồ khối hàm char line()

Lưu ý: dấu tích trong sơ đồ biểu thị cho việc kiểm tra điều kiện đã đúng, dấu x là biểu thị cho việc kiểm tra điều kiện đã sai.

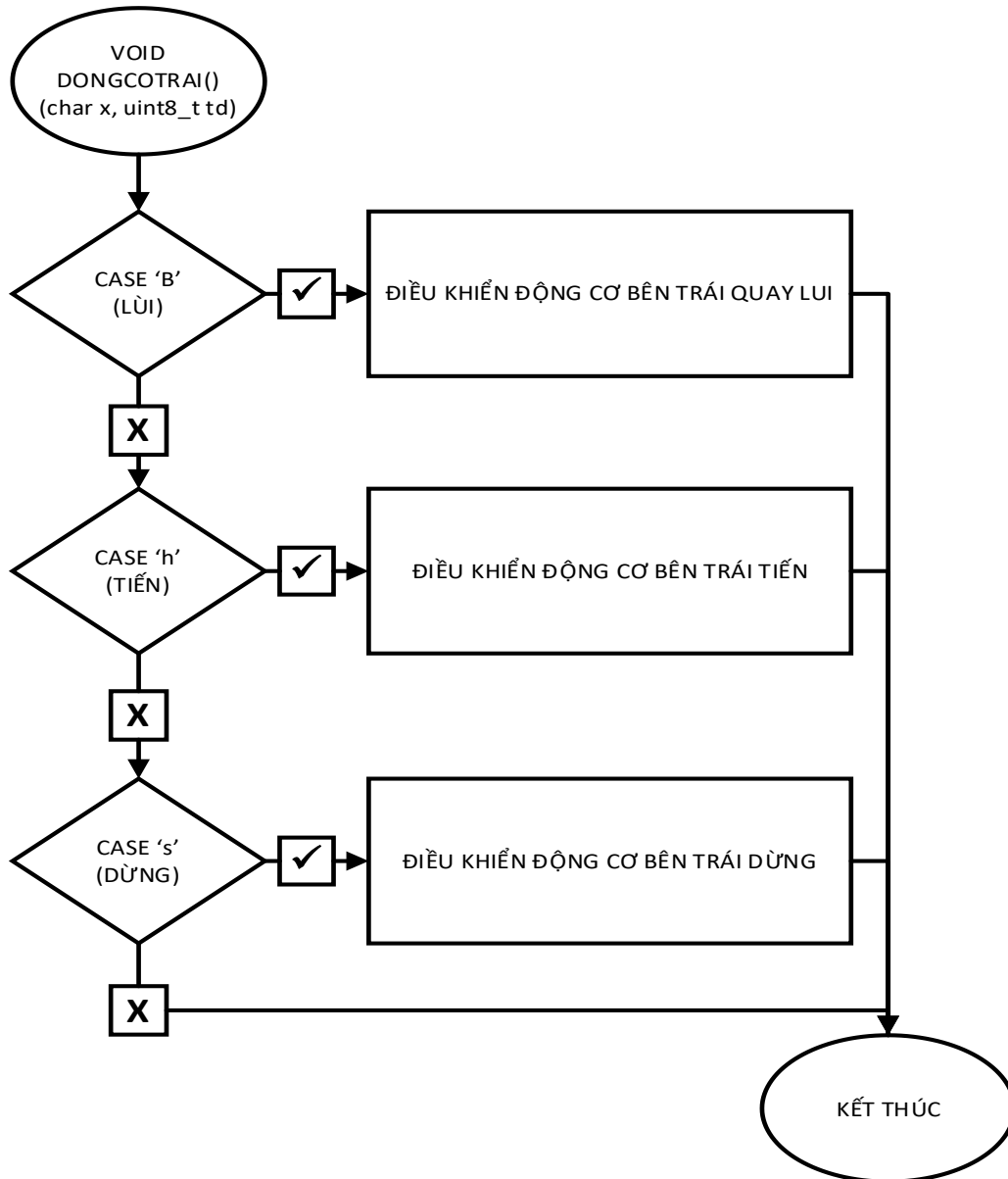
Hàm này có chính năng là giúp xe có thể đi theo line bằng cách trả về các giá trị.

‘r’: chạy hướng về phải

‘h’: đi thẳng

‘l’: chạy hướng về trái

2.2.3.9 Sơ đồ khối Hàm void dongcotrai(char x, unsigned int td):



Hình 2.11 Sơ đồ khối hàm void dongcotrai()

Lưu ý: dấu tích trong sơ đồ biểu thị cho việc kiểm tra điều kiện đã đúng, dấu x là biểu thị cho việc kiểm tra điều kiện đã sai.

Hàm này có chức năng sẽ điều khiển động cơ bên trái. Việc viết hàm này giúp tiết kiệm thời gian trong quá trình lập trình.

Hàm sẽ nhận giá trị hành động và tốc độ từ bên trong hàm dongco() sẽ được đề cập ở mục 2.2.3.11. Từ đó dựa vào cấu trúc switch case và thông qua câu lệnh digitalWrite và analogWrite để điều khiển động cơ theo ý muốn.

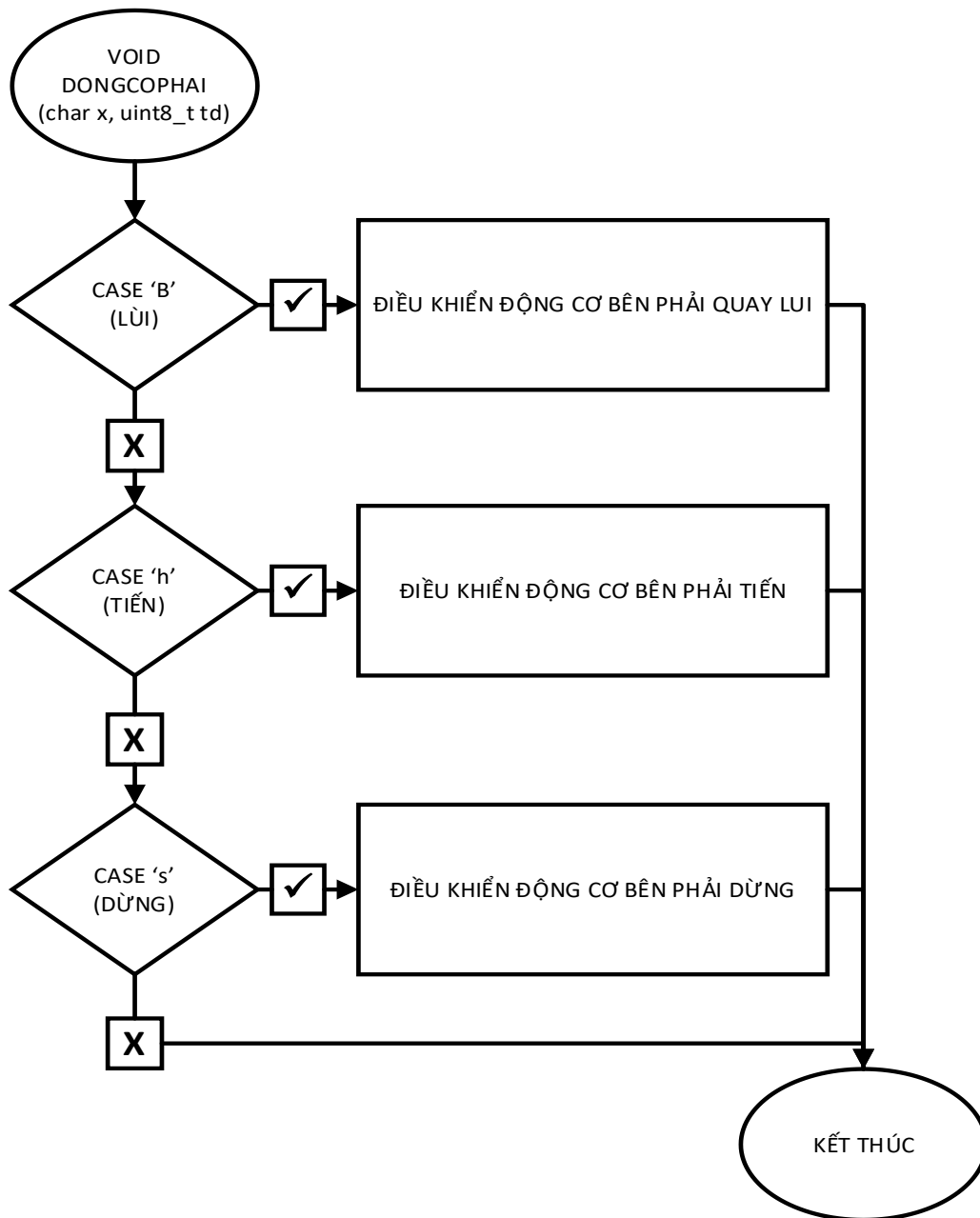
Các trường hợp switch case:

‘b’: lui

‘h’: tiến

‘s’: dừng lại

2.2.3.10 Sơ đồ khối Hàm void dongcophai(char x, unsigned int td):



Hình 2.12 Sơ đồ khối hàm `dongcophai()`

Lưu ý: dấu tích trong sơ đồ biểu thị cho việc kiểm tra điều kiện đã đúng, dấu x là biểu thị cho việc kiểm tra điều kiện đã sai.

Hàm này có chức năng sẽ điều khiển động cơ bên phải. Việc viết hàm này giúp tiết kiệm thời gian trong quá trình lập trình.

Hàm sẽ nhận giá trị hành động và tốc độ từ bên trong hàm `dongco()` sẽ được đề cập ở mục 2.2.3.11. Từ đó dựa vào cấu trúc switch case và thông qua câu lệnh `digitalWrite` và `analogWrite` để điều khiển động cơ theo ý muốn.

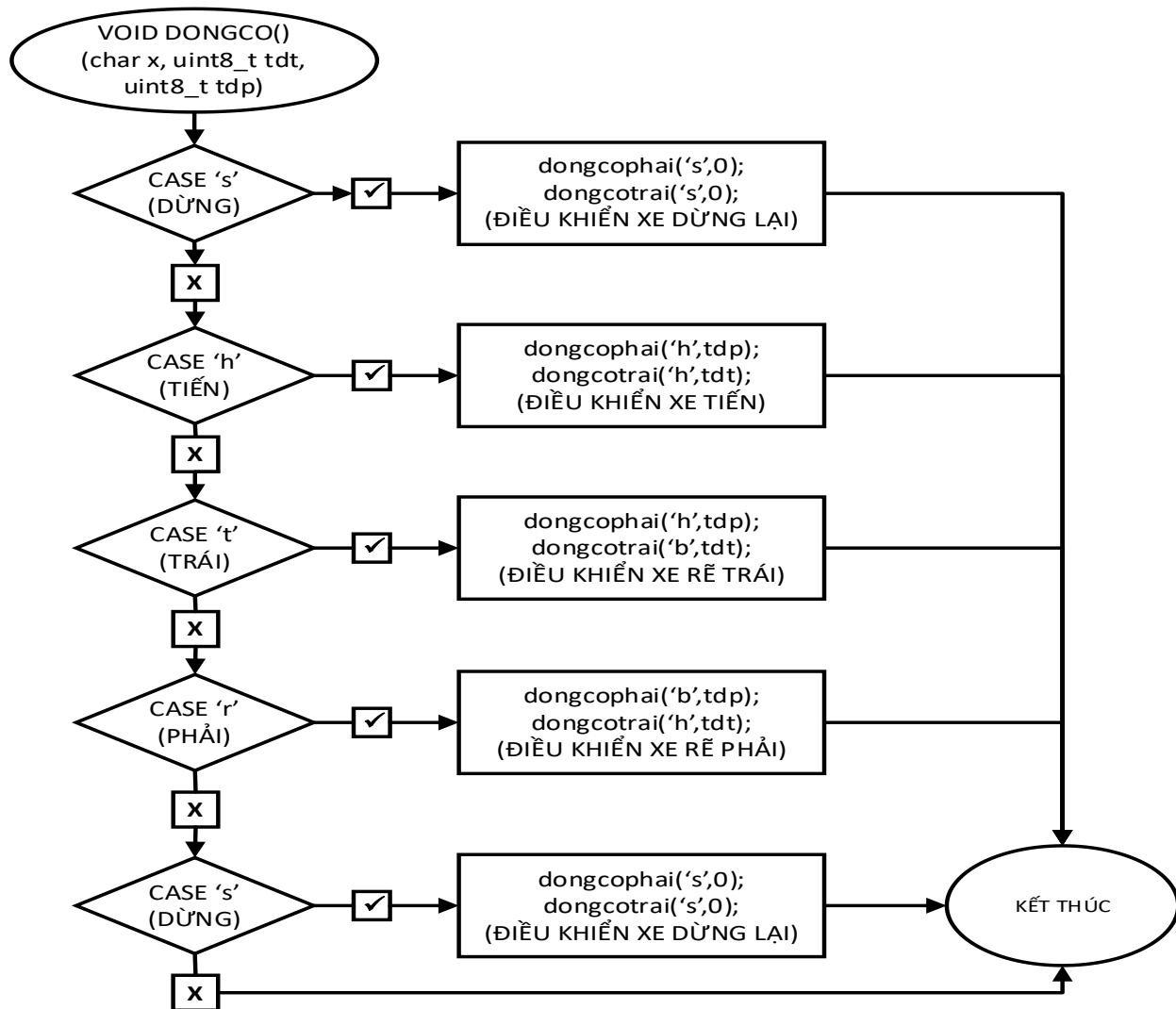
Các trường hợp switch case:

‘b’: lui

‘h’: tiến

‘s’: dừng lại

2.2.3.11 Sơ đồ khối Hàm void dongco(char x, unsigned int tdt, unsigned int tdp):

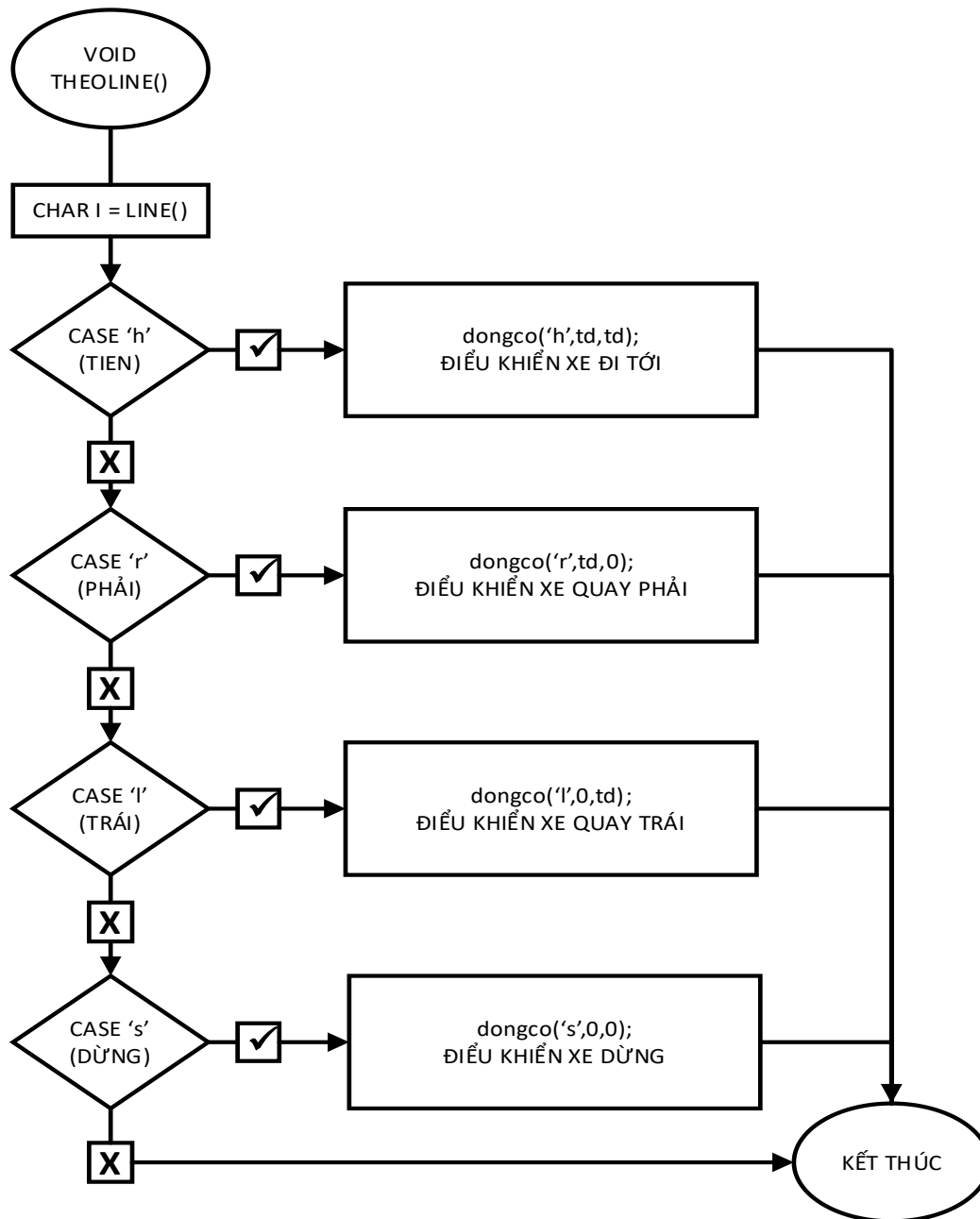


Hình 2.13 Sơ đồ khối hàm dongco()

Lưu ý: dấu tích trong sơ đồ biểu thị cho việc kiểm tra điều kiện đã đúng, dấu x là biểu thị cho việc kiểm tra điều kiện đã sai.

Hàm này có tác dụng điều khiển động cơ xe theo ý muốn thông qua hàm dongcophai() và dongcotrai() được đề cập ở mục 2.2.3.9 và 2.2.3.10 và cấu trúc Switch case. Việc viết hàm này nhằm tiết kiệm thời gian trong quá trình lập trình.

2.2.3.12 Sơ đồ khối Hàm void theoline():

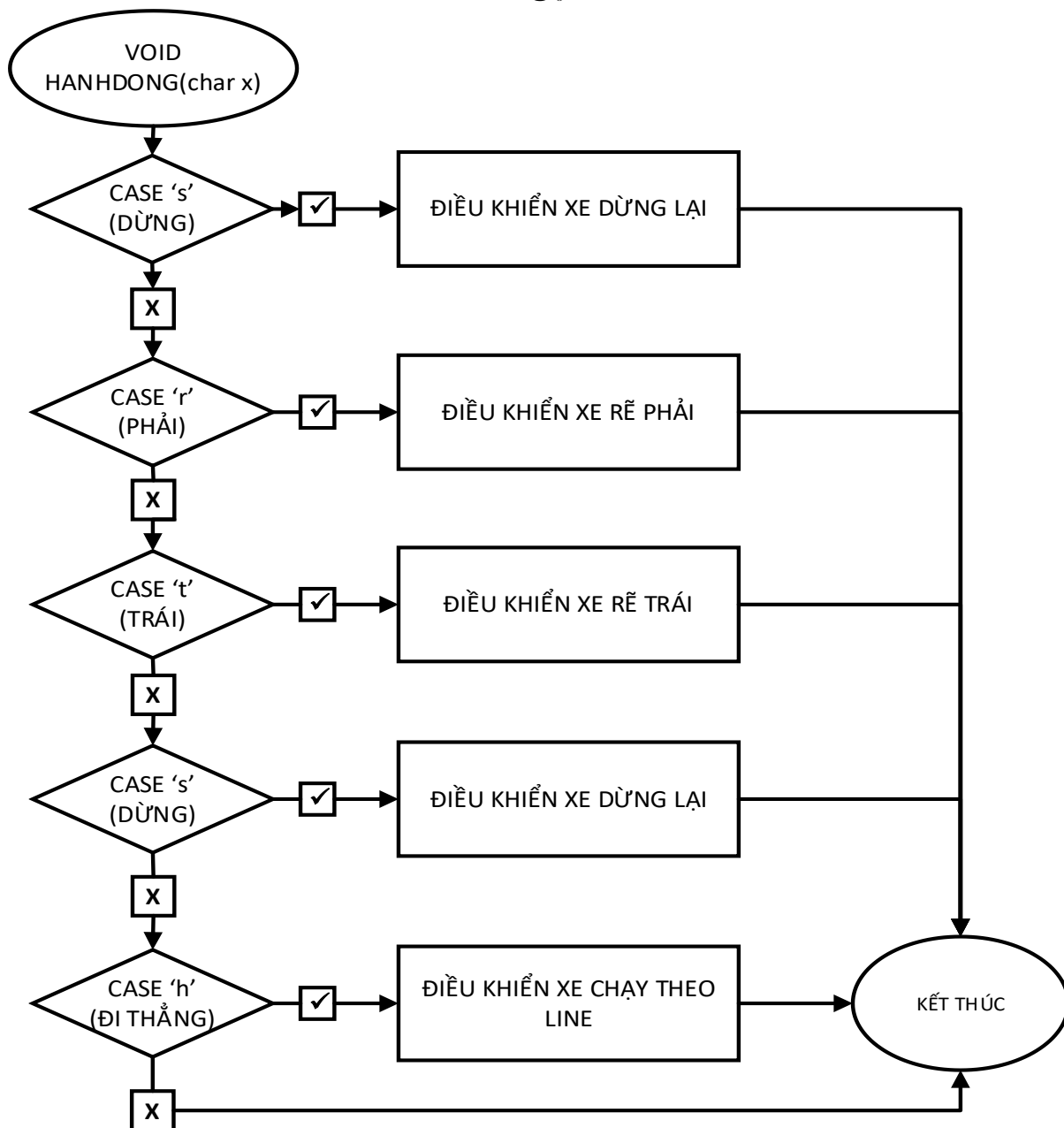


Hình 2.14 Sơ đồ khối theoline()

Lưu ý: dấu tích trong sơ đồ biểu thị cho việc kiểm tra điều kiện đã đúng, dấu x là biểu thị cho việc kiểm tra điều kiện đã sai.

Hàm này có chức năng dựa vào giá trị của biến char l được nhận từ hàm line() được đề cập ở mục 2.2.3.8. Rồi thông qua cấu trúc đưa ra các giá trị để vào hàm hanhdong() từ đó có thể điều khiển xe theo ý muốn.

2.2.3.13 Sơ đồ khối Hàm void hanhdong():

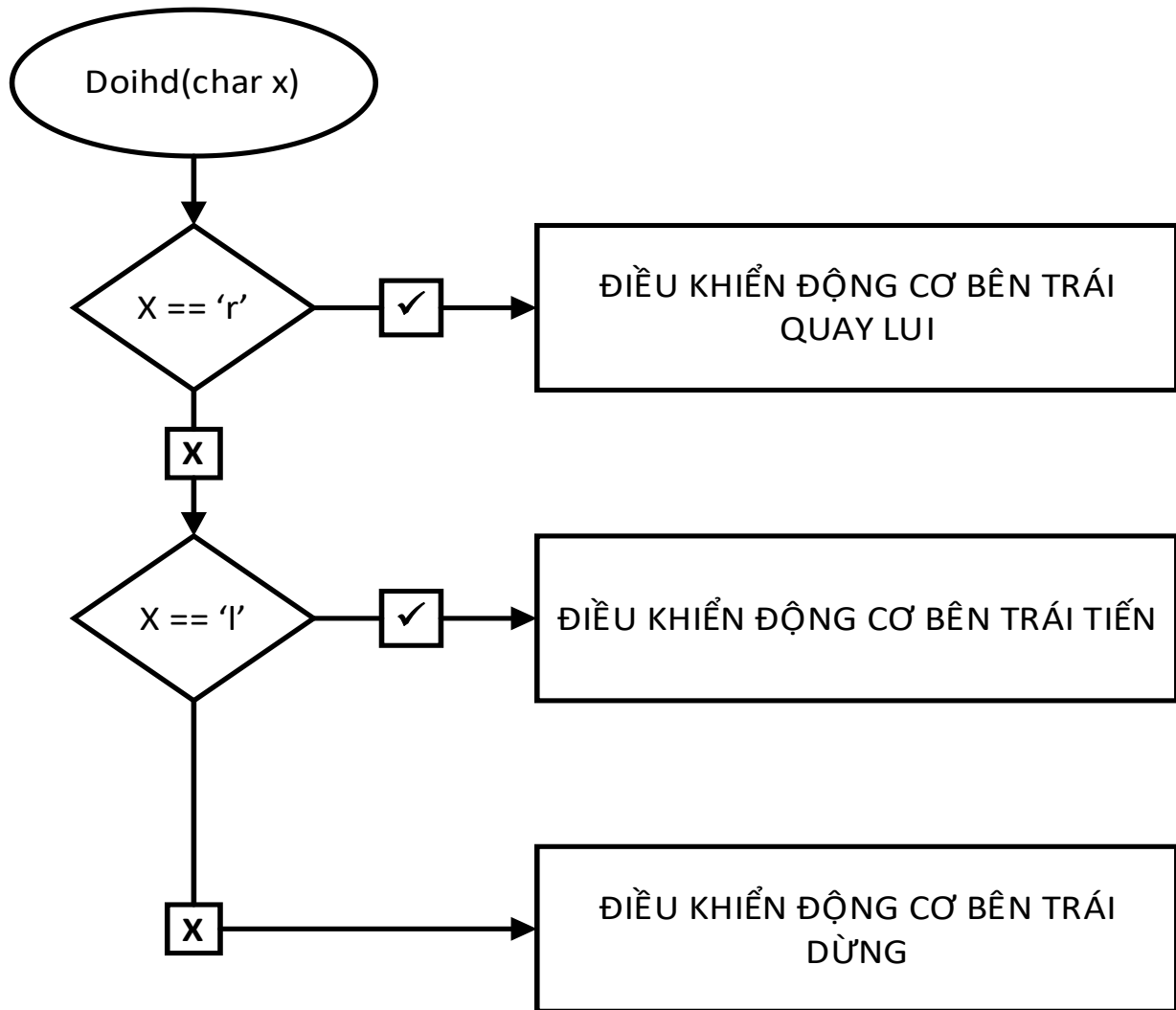


Hình 2.15 Sơ đồ khối hàm hanhdong()

Lưu ý: dấu tích trong sơ đồ biểu thị cho việc kiểm tra điều kiện đã đúng, dấu x là biểu thị cho việc kiểm tra điều kiện đã sai.

Hàm này có chức năng chính là nhận giá trị được đưa vào và phân giải giá trị đó để đưa vào hàm dongco() kết hợp với câu lệnh delay() và cấu trúc switch case cũng như cấu trúc while. Kết hợp những thứ đó lại giúp cho xe thực hiện các hành động thực tế như quẹo phải, quẹo trái, quay đầu, hoặc ra lệnh đi theo line cũng như dừng lại.

2.2.3.14 Sơ đồ khối Hàm char doihd():



Hình 3.16 Sơ đồ khối hàm doihd()

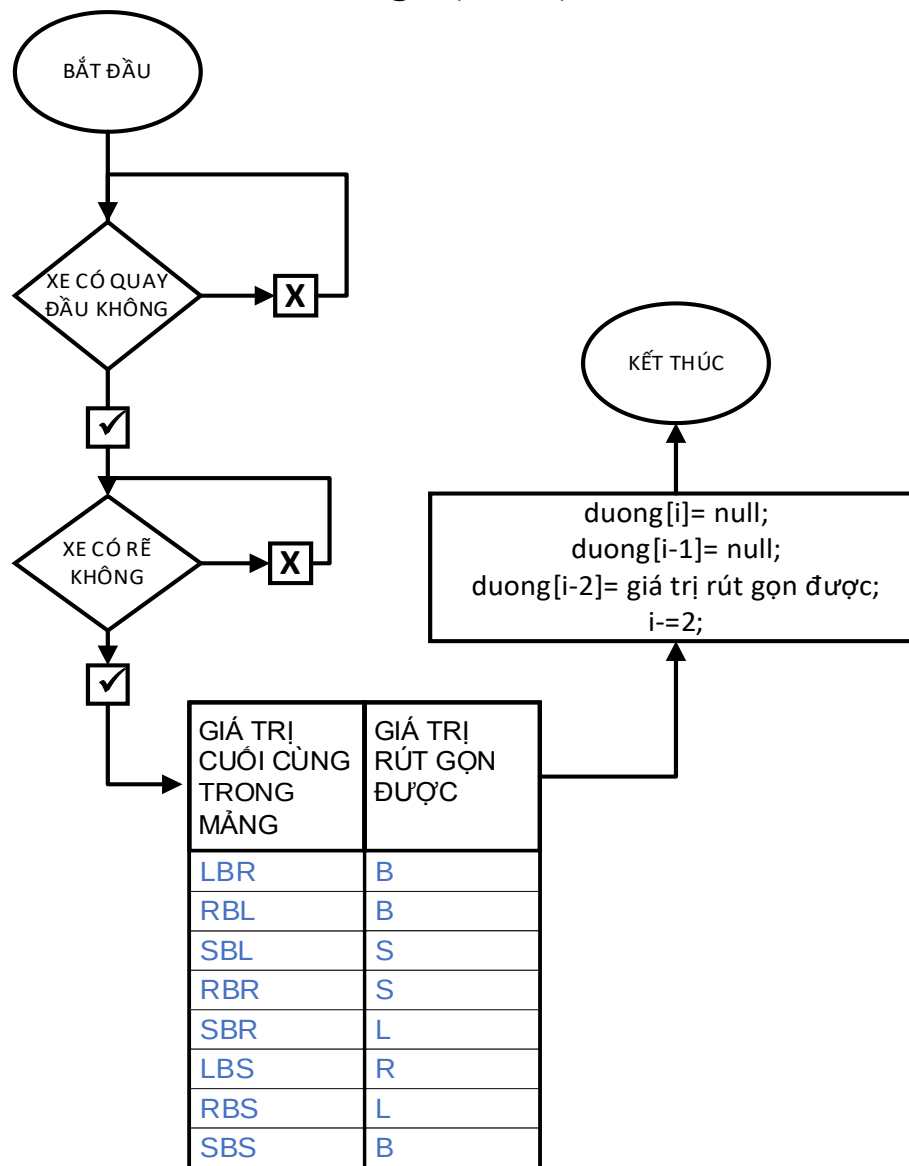
Chức năng của hàm này là giúp xe khi quay đầu lại đi đến đích đến. Thì phải đổi ngược lại hành động được lưu vào mảng nhớ.

Quy tắc đổi như sau:

'l' -> 'r'

'r' -> 'l'

2.2.3.15 Sơ đồ khối Hàm void rutngan(char x):



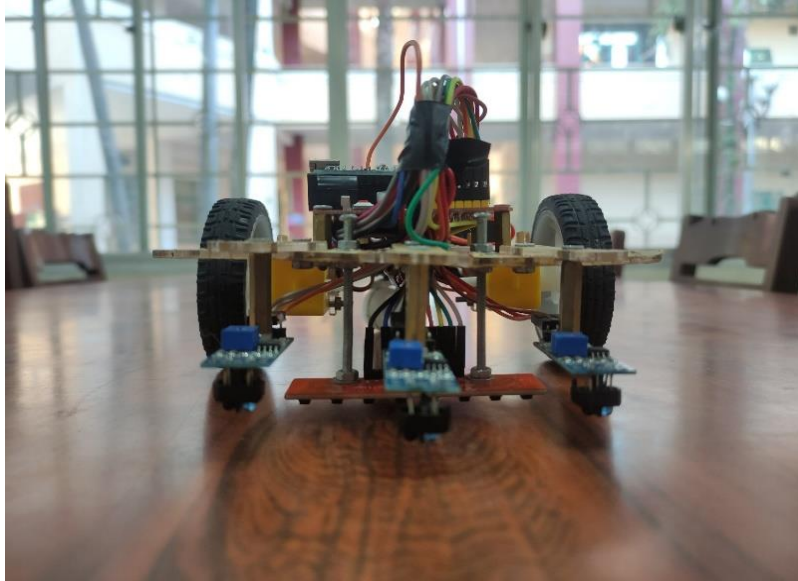
Hình 3.17 Sơ đồ khối hàm rutngan()

Hàm này có tác dụng khi xe lưu các điểm có trên line vào trong mảng. Hàm này sẽ lấy 3 giá trị mới nhất từ các điểm đó so sánh nếu trùng với các giá trị giống sơ đồ thì sẽ xử lý và rút về giá trị ở cột bên phải. Từ đó xe chỉ lưu lại các điểm đi đến đích mà không lưu lại các điểm chỉ vào đường cụt.

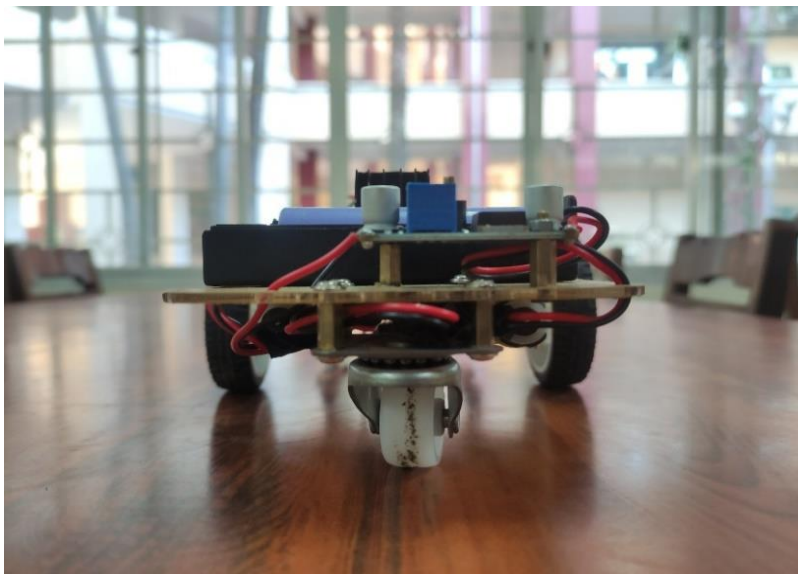
CHƯƠNG 3: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

3.1 Hình ảnh của xe:

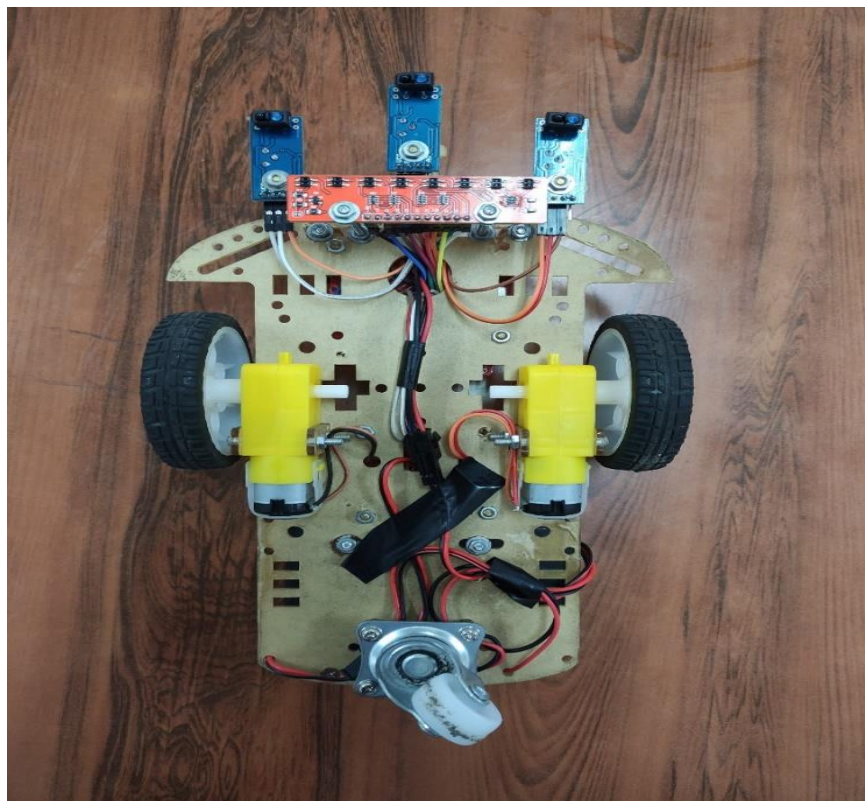
3.1.1 Hình ảnh xe sau khi hoàn thành:



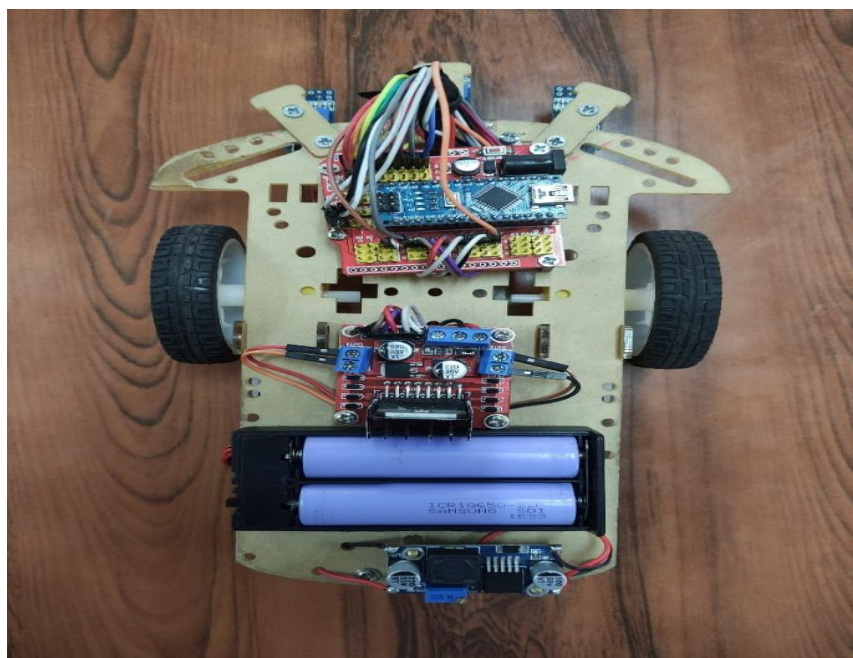
Hình 3.1 Mặt trước của xe



Hình 3.2 Mặt sau của xe



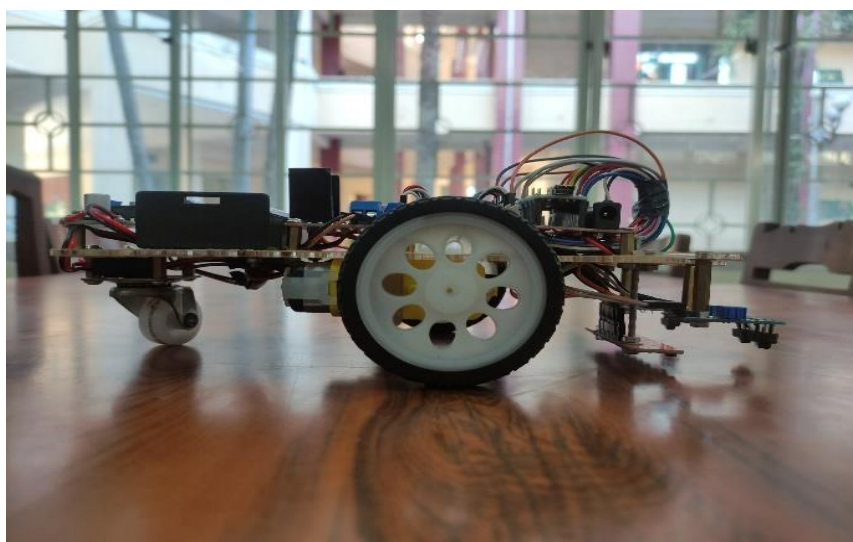
Hình 3.3 mặt dưới của xe



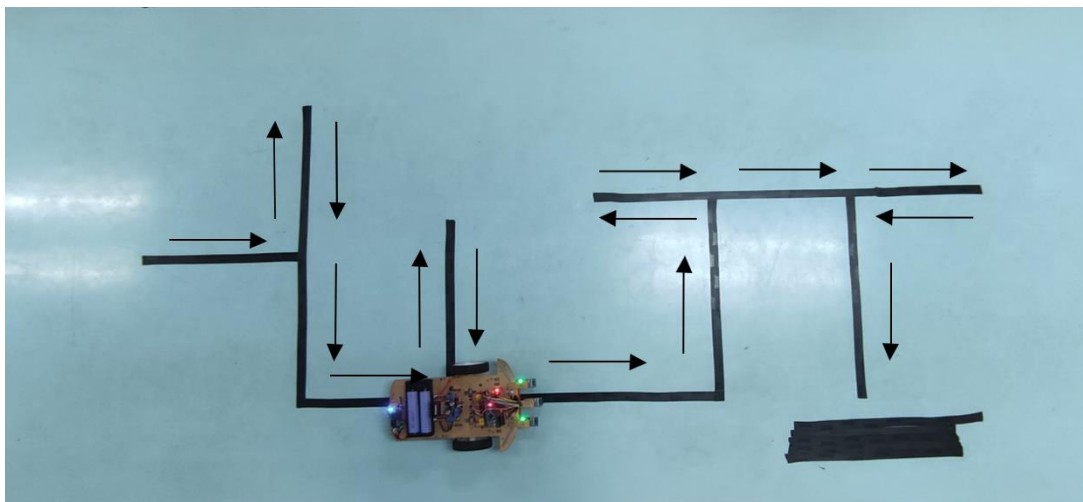
Hình 3.4 Mặt trên của xe



Hình 3.5 Mặt bên trái của xe



Hình 3.6 Mặt bên phải của xe



3.7 Hình ảnh xe trong mê cung line

3.2 Kết quả đạt được:

3.2.1 Thành viên:

Được tiếp xúc, trải nghiệm làm nên sản phẩm thực tế, nâng cao kiến thức và tập làm báo cáo.

Nâng cao kỹ năng làm việc nhóm của từng thành viên.

3.2.2 Sản phẩm:

Sản phẩm đơn giản giá thành rẻ.

Dễ nhìn

3.2.3 Đánh giá sản phẩm:

Sản phẩm có kết cấu chưa được vững chắc, điều này có thể làm cho xe bị lỗi trong quá trình giải mã mê cung.

Xe di chuyển còn rung lắc nhẹ và chậm.

Thuật toán phân biệt line chưa được chính xác một cách tuyệt đối còn có thể xảy ra nhầm lẫn trong việc phân biệt line.

Thuật toán chỉ dừng lại ở việc giải mã mê cung chứ không phải tìm đường ngắn nhất.

3.2.4 Hướng Phát Triển:

Phát triển thành con robot có khả năng vận chuyển đồ vật có trọng tải lớn di chuyển trong các bãi vận chuyển, giúp giảm thiểu sức lao động của con người, tăng độ chính xác, giảm thiểu chi phí thuê nhân công.

Nguồn tham khảo

- [1] “arduino,” 5 2 2018. [Trực tuyến]. Available: <https://www.arduino.cc/en/Guide/Introduction>.
- [2] “store-usa.arduino,” [Trực tuyến]. Available: <https://store-usa.arduino.cc/products/arduino-nano?selectedStore=us>.
- [3] <https://www.thegioiic.com/arduino-nano-atmega328-v3>, “Arduino Nano ATMEGA328 V3,” [Trực tuyến]. Available: <https://www.thegioiic.com/arduino-nano-atmega328-v3>.
- [4] “pololu,” [Trực tuyến]. Available: <https://www.pololu.com/product/960>.
- [5] “thegioiic,” [Trực tuyến]. Available: <https://www.thegioiic.com/tcrt5000-cam-bien-do-line-khoang-cach-0-3cm-ngo-ra-ttl>.
- [6] “thietbikythuat,” 18 1 2021. [Trực tuyến]. Available: <https://thietbikythuat.com.vn/nguyen-ly-hoat-dong-cua-cam-bien-hong-ngoai-la-gi/>.
- [7] instructable_translator, “arduino.vn,” 16 6 2016. [Trực tuyến]. Available: <http://arduino.vn/bai-viet/893-cach-dung-module-dieu-khien-dong-co-l298n-cau-h-de-dieu-khien-dong-co-dc>.
- [8] “thegioiic,” [Online]. Available: <https://www.thegioiic.com/lm2596s-mach-giam-ap-3a>.
- [9] “thegioiic,” [Online]. Available: <https://www.thegioiic.com/mach-mo-rong-arduino-nano-shield-v3-0>.
- [10] Mjrovai, “instructables,” 28 9 2017. [Trực tuyến]. Available: <https://www.instructables.com/Maze-Solver-Robot-Using-Artificial-Intelligence-Wi/>.

Phần Phụ Lục:

```
uint8_t cambien[11] = { A0, A1, A2, A3, A4, A5, A6, A7, 3, 4, 11 };
```

```
#define dct1 5 // động cơ bên trái tiền (1)
```

```
#define dct2 6 // động cơ bên trái lui (1)
```

```
#define dct 9 // tốc độ động cơ trái
```

```
#define dcp1 7 //động cơ bên phải tiền (1)
```

```
#define dcp2 8 //động cơ bên phải lui (1)
```

```
#define dcp 10 //tốc độ động cơ phải
```

```
uint8_t cb[11], co = 0;
```

```
char danhdauduong[30], giatriphandoan;
```

```
int d1 = 0, d2 = 0;
```

```
int trangthai = 1;
```

```
const int TOCDO = 97;
```

```
void indongco() {
```

```
    pinMode(dcp1, OUTPUT);
```

```
    pinMode(dcp2, OUTPUT);
```

```
    pinMode(dcp, OUTPUT);
```

```
    pinMode(dct1, OUTPUT);
```

```
    pinMode(dct2, OUTPUT);
```

```
    pinMode(dct, OUTPUT);
```

```
}
```

```
void incambien() {
```

```
    digitalWrite(2, HIGH);
```

```

pinMode(cambien[0], INPUT);
pinMode(cambien[1], INPUT);
pinMode(cambien[2], INPUT);
pinMode(cambien[3], INPUT);
pinMode(cambien[4], INPUT);
pinMode(cambien[5], INPUT);
pinMode(cambien[6], INPUT);
pinMode(cambien[7], INPUT);
pinMode(cambien[8], INPUT);
pinMode(cambien[9], INPUT);
pinMode(cambien[10], INPUT);
}

```

```

uint8_t digital(uint8_t x) {
    if (analogRead(x) > 890)
        return 1;
    else
        return 0;
}

```

```

void resetcb() {
    cb[9] = digitalRead(cambien[9]);
    cb[10] = digitalRead(cambien[10]);
    cb[8] = digitalRead(cambien[8]);
    cb[0] = digital(cambien[0]);
    cb[1] = digital(cambien[1]);
    cb[2] = digital(cambien[2]);
}

```

```

cb[3] = digital(cambien[3]);
cb[4] = digital(cambien[4]);
cb[5] = digital(cambien[5]);
cb[6] = digital(cambien[6]);
cb[7] = digital(cambien[7]);
}

char phandoan() {
    resetcb();

    if (!cb[7] && !cb[6] && !cb[5] && !cb[4] && !cb[3] && !cb[2] && !cb[1] && !cb[0]
    && cb[9])

        return 's';

    if ((cb[8] && cb[10]) || (cb[7] && cb[6] && cb[5] && cb[4] && cb[3] && cb[2] &&
    cb[1] && cb[0])) {

        resetcb();

        if (!cb[7] && !cb[6] && !cb[5] && !cb[4] && !cb[3] && !cb[2] && !cb[1] &&
        !cb[0] && cb[9])

            return 's';

        dongco('h', TOCDO, TOCDO);

        delay(240);

        resetcb();

        if (cb[0] || cb[1] || cb[2] || cb[3] || cb[4] || cb[5] || cb[6] || cb[7] || cb[9])

            return 'f';

        else

            return 't';

    }

    if (cb[7] && cb[6] && cb[5] && cb[4] && !cb[1] && !cb[0]) {

        dongco('h', TOCDO, TOCDO);
    }
}

```



```

    delay(170);
    resetcb();
    if (cb[0] || cb[1] || cb[2] || cb[3] || cb[4] || cb[5] || cb[6] || cb[7] || cb[9])
        return 'a';
    else
        return 'l';
}

if (cb[0] && cb[1] && cb[2] && cb[3] && !cb[6] && !cb[7]) {
    dongco('h', TOCDO, TOCDO);
    delay(170);
    resetcb();
    if (cb[0] || cb[1] || cb[2] || cb[3] || cb[4] || cb[5] || cb[6] || cb[7] || cb[9])
        return 'd';
    else
        return 'r';
}

if (!cb[7] && !cb[6] && !cb[5] && !cb[4] && !cb[3] && !cb[2] && !cb[1] && !cb[0]
&& !cb[8] && !cb[9] && !cb[10]) {
    dongco('h', TOCDO, TOCDO);
    delay(200);
    resetcb();

    if (!cb[7] && !cb[6] && !cb[5] && !cb[4] && !cb[3] && !cb[2] && !cb[1] &&
!cb[0] && !cb[8] && !cb[9] && !cb[10])
        return 'o';
    } else
        return 'h'; // theo line
}

```

```

char line() {
    if ((cb[0] || cb[1] || cb[2] || cb[3]) && !cb[4] && !cb[5] && !cb[6] && !cb[7])
        return 'r';

    else if (!cb[0] && !cb[1] && !cb[2] && cb[3] && cb[4] && !cb[5] && !cb[6] &&
!cb[7])
        return 'h';

    else if (!cb[0] && !cb[1] && !cb[2] && !cb[3] && (cb[4] || cb[5] || cb[6] || cb[7]))
        return 'l';

}

```

```

void dongcotrai(char x, unsigned int TOCDO) {
    switch (x) {
        case 'b': // động cơ trái lui
            digitalWrite(dct1, HIGH);
            digitalWrite(dct2, LOW);
            analogWrite(dct, TOCDO);
            break;

        case 'h': // động cơ trái tiến
            digitalWrite(dct1, LOW);
            digitalWrite(dct2, HIGH);
            analogWrite(dct, TOCDO);
            break;

        case 's': // động cơ trái dừng
            digitalWrite(dct1, HIGH);

```

```

    digitalWrite(dct2, HIGH);
    analogWrite(dct, 0);
    break;
}
}

void dongcophai(char x, unsigned int TOCDO) {
    switch (x) {
        case 'b': // động cơ phải lui
            digitalWrite(dcp1, HIGH);
            digitalWrite(dcp2, LOW);
            analogWrite(dcp, TOCDO);
            break;

        case 'h': // động cơ phải tiến
            digitalWrite(dcp1, LOW);
            digitalWrite(dcp2, HIGH);
            analogWrite(dcp, TOCDO);
            break;

        case 's': // động cơ phải dừng
            digitalWrite(dcp1, HIGH);
            digitalWrite(dcp2, HIGH);
            analogWrite(dcp, 0);
            break;
    }
}

```

```

void dongco(char x, unsigned int tdt, unsigned int tdp) {
    switch (x) {
        case 's': // xe dừng
            dongcophai('s', 0);
            dongcotrai('s', 0);
            break;

        case 'h': // xe tiến
            dongcophai('h', tdp);
            dongcotrai('h', tdt);
            break;

        case 'l': // xe quẹo trái
            dongcophai('h', tdp);
            dongcotrai('b', tdt);
            break;

        case 'r': // xe quẹo phải
            dongcophai('b', tdp);
            dongcotrai('h', tdt);
            break;

        case 'b': // xe lui
            dongcophai('b', tdp);
            dongcotrai('b', tdt);
            break;
    }
}

```

```
}  
}
```

```
void theoline() {  
    char l = line();  
    switch (l) {  
        case 'h': // tiến  
            dongco('h', TOCDO, TOCDO);  
            break;  
  
        case 'r': // nhích phải  
            dongco('r', TOCDO, 0);  
            break;  
  
        case 'l': // nhích trái  
            dongco('l', 0, TOCDO);  
            break;  
  
        case 's': // dừng  
            dongco('s', 0, 0);  
            break;  
    }  
}
```

```
void hanhdong(char x) {  
    switch (x) {  
        case 'l':
```

```
    dongco('l', TOCDO, TOCDO);  
    delay(300);  
    do {  
        dongco('l', TOCDO, TOCDO);  
        resetcb();  
    } while (!(cb[3] && cb[4]));  
    break;
```

```
case 'r':  
    dongco('r', TOCDO, TOCDO);  
    delay(350);  
    do {  
        dongco('r', TOCDO, TOCDO);  
        resetcb();  
    } while (!(cb[4] && cb[3]));  
    break;
```

```
case 'b':  
    do {  
        dongco('r', 120, 120);  
        resetcb();  
    } while (!(cb[3] && cb[4]));  
    break;
```

```
case 's':  
    dongco('s', TOCDO, TOCDO);  
    break;
```

```

    case 'h':
        theoline();
        break;
    }
}

```

```

char doihd(char x) {
    if (x == 'r')
        return 'l';
    else if (x == 'l')
        return 'r';
    else
        return x;
}

```

```

void rutngan(char x) {
    dongco('s', 0, 0);
    if (x == 'd') {
        if (co == 1)
            ++co;
        danhdauduong[d1] = 'h';
        d1++;
    } else if (x == 'o') {
        ++co;
        danhdauduong[d1] = 'b';
        d1++;
    }
}

```

```

} else {
    if (co == 1)
        ++co;
    danhdauduong[d1] = 'l';
    d1++;
}
if (co == 2) {
    if (danhdauduong[d1 - 3] == 'l') {
        if (danhdauduong[d1 - 1] == 'r') {
            danhdauduong[d1 - 3] = 'b';
            danhdauduong[d1 - 2] = '\0';
            danhdauduong[d1 - 1] = '\0';
            d1 += -2;
            co = 1;
        } else if (danhdauduong[d1 - 1] == 'h') {
            danhdauduong[d1 - 3] = 'r';
            danhdauduong[d1 - 2] = '\0';
            danhdauduong[d1 - 1] = '\0';
            d1 += -2;
            co = 0;
        } else if (danhdauduong[d1 - 1] == 'l') {
            danhdauduong[d1 - 3] = 'h';
            danhdauduong[d1 - 2] = '\0';
            danhdauduong[d1 - 1] = '\0';
            d1 += -2;
            co = 0;
        }
    }
}

```



```

} else if (danhdauduong[d1 - 3] == 'h') {
    if (danhdauduong[d1 - 1] == 'l') {
        danhdauduong[d1 - 3] = 'r';
        danhdauduong[d1 - 2] = '\0';
        danhdauduong[d1 - 1] = '\0';
        d1 += -2;
        co = 0;
    } else if (danhdauduong[d1 - 1] == 'h') {
        danhdauduong[d1 - 3] = 'b';
        danhdauduong[d1 - 2] = '\0';
        danhdauduong[d1 - 1] = '\0';
        d1 += -2;
        co = 1;
    }
} else if (danhdauduong[d1 - 3] == 'r') {
    if (danhdauduong[d1 - 1] == 'l') {
        danhdauduong[d1 - 3] = 'b';
        danhdauduong[d1 - 2] = '\0';
        danhdauduong[d1 - 1] = '\0';
        d1 += -2;
        co = 1;
    }
}
}
}
}

```

```

void setup() {
    indongco();
    incambien();
}

void loop() {
    if (trangthai == 1) {
        giatriphandoan = phandoan();
        switch (giatriphandoan) {
            case 'l':
                hanhdong('l');
                break;

            case 'r':
                hanhdong('r');
                break;

            case 'a':
                rutngan('a');
                hanhdong('l');
                break;

            case 'd':
                rutngan('d');
                hanhdong('h');
                break;

            case 'f':

```

```

        rutngan('f');
        hanhdong('l');
        break;

    case 't':
        rutngan('t');
        hanhdong('l');
        break;

    case 'o':
        rutngan('o');
        hanhdong('b');
        break;

    case 's':
        trangthai++;
        hanhdong('s');
        d2 = d1 - 1;
        delay(3000);
        hanhdong('b');
        break;

    case 'h':
        theoline();
        break;
}
} else if (trangthai == 2) {

```

```
giatriphandoan = phandoan();
switch (giatriphandoan) {
    case 'l':
        hanhdong('l');
        break;

    case 'r':
        hanhdong('r');
        break;

    case 'a':
        hanhdong(doihd(danhdauduong[d2]));
        d2 -= 1;
        break;

    case 'd':
        hanhdong(doihd(danhdauduong[d2]));
        d2 -= 1;
        break;

    case 'f':
        hanhdong(doihd(danhdauduong[d2]));
        d2 -= 1;
        break;

    case 't':
        hanhdong(doihd(danhdauduong[d2]));
```

```

    d2 -= 1;
    break;

case 'o':
    trangthai++;
    d2 = 0;
    hanhdong('s');
    delay(3000);
    hanhdong('b');
    break;

case 'h':
    theoline();
    break;
}
} else if (trangthai == 3) {
    giatriphandoan = phandoan();
    switch (giatriphandoan) {
        case 'l':
            hanhdong('l');
            break;

        case 'r':
            hanhdong('r');
            break;

        case 'a':

```

```
hanhdong(danhdauduong[d2]);  
++d2;  
break;
```

```
case 'd':  
    hanhdong(danhdauduong[d2]);  
    ++d2;  
    break;
```

```
case 'f':  
    hanhdong(danhdauduong[d2]);  
    ++d2;  
    break;
```

```
case 't':  
    hanhdong(danhdauduong[d2]);  
    ++d2;  
    break;
```

```
case 'o':  
    hanhdong('b');  
    break;
```

```
case 's':  
    trangthai++;  
    hanhdong('s');  
    break;
```

```
case 'h':  
    theoline();  
    break;  
}  
} else {  
    dongco('s', 0, 0);  
}  
}
```

Ngày hội thảo: 15/11/2023, Ngày nghiệm thu: 27/11/2023